

Auckland University of Technology

ENSE707 - Software Quality Assurance

Final Project Report

Group: Software Quality Assurance
Thi Mai Phuong Nguyen - 23197030
Benjamin Macpherson - 23208226
Thanh Tran - 24247169

18/10/2026

Table of Contents

1. Introduction 5
2. Project Completion and Final Implementation 6
   2.1. Completed Features 6
   2.2. Functional Requirements and Acceptance Criteria 6
   2.3. Changes to Scope, Architecture, and Technology 8
      i. Changes to Scope 8
      ii. Changes to Architecture (Information Architecture/Design) 8
      iii. Changes to Technology 8
      iv. Visual Change Log 9
3. Requirements Verification and Traceability 16
   3.1. Review and Finalisation of Requirements 16
   3.2. Updated Requirements Traceability Matrix 17
   3.3. Requirements Not Fully Verified or Out of Scope 21
4. Test Execution and Final Test Results 22
   4.1. Test Execution 22
   4.2. Final Test Cases and Execution Results 22
      i. Automated Test Results 23
      ii. Manual Test Results 24
   4.3. Test Execution Summary 31
   4.4. Failed Tests and Corrective Actions 31
   4.5. Regression Testing 32
   4.6. Remaining Quality Risks 32
5. Test Automation and Quality Engineering 33
   5.1. Test Case Selection for Automation 33
   5.2. Implementation of Automated Tests 33
   5.3. Automated Test Execution Results 33
   5.4. Benefits, Limitations, and Maintainability 33
   5.5. Justification for Manual and Exploratory Testing 34
6. Continuous Integration and DevOps QA 35
   6.1. Development and Integration Workflow 35
   6.2. CI Implementation 35
   6.3. Quality Gates 35

6.4. Smoke and Sanity Testing 36
6.5. Limitations and Future Improvements 41
7. Defect Management and Root Cause Analysis 42
7.1. Defect Management Approach 42
7.2. Final Defect Register 42
7.3. Root Cause Analysis 44
i. DEF-01: Booking Validation and Database Integration 44
ii. DEF-02: Ambiguous Vietnamese Translation 44
iii. DEF-03: Application Startup and Database Configuration 45
7.4. Defect Lifecycle 45
7.5. Corrective Actions and Defect Prevention 46
7.6. Evaluation of Defect Management 47
8. Quality Testing 48
8.1. Performance Quality Testing 48
8.2. Security Quality Testing 50
9. Quality Metrics and Final Release Decision 53
9.1. Test Pass Rate 53
9.2. Defect Severity Distribution 54
9.3. Requirement Coverage 54
9.4. Test Coverage 55
9.5. Automation coverage 55
9.6. Code Quality Indicators 55
9.7. Commit Activity 57
9.8. Evaluation 58
9.9. Release Decision 58
10. AI-Assisted Development and Critical Reflection 59
10.1. How GitHub Copilot Was Used 59
10.2. How AI Outputs Were Reviewed 66
10.3. Quality Risks Found in AI-Assisted Output 66
10.4. Reflection 67
11. Quality Improvement and Future Recommendations 68
11.1. Main Quality Lessons Learned 68
11.2. Recommended Improvements to the Development and QA Process 68
11.3. Future Improvements 69

11.4. Evolving Towards Continuous Quality Engineering 70
12. Conclusion 71
Appendix A: Team Contribution Statement 72

1. Introduction

Software Quality Assurance (SQA) is important to ensure that software systems meet functional requirements while maintaining acceptable standards for reliability, availability, security, performance, and maintainability. This project continues the development and quality improvement of the Study Room Booking and Management System from Assessment 1, which aims to address the inefficiencies of manual study room booking processes in universities. The proposed system provides a centralised platform for searching, booking, modifying, and cancelling room reservations, as well as managing the system itself. This helps minimise booking conflicts, reduces administrative workload, and improves the visibility of room availability.

Assessment 1 had defined the problem, stakeholder requirements, software quality attributes, initial prototype, testing strategy, and Requirements Traceability Matrix. The initial prototype demonstrated core booking functionality and utilised MSTest to verify selected business rules. However, there were still some limitations, including the use of in-memory repositories, incomplete administrative functions, limited testing scope, and insufficient evaluation of non-functional requirements. Assessment 2 focuses on addressing these limitations by completing the implementation, expanding the test coverage, as well as evaluating the system against the established requirements and acceptance criteria.

The project applies systematic QA practices throughout the final development phase, including requirements verification and traceability, automated testing, regression testing, defect management, root cause analysis, and quality measurement. Functional testing is used to verify key booking workflows and validation rules, while non-functional testing evaluates relevant quality attributes. After testing, the test results and defect records provide evidence for assessing the effectiveness of corrective actions, identifying residual risks, and determining whether the final prototype meets its intended objectives.

GitHub supports version control, collaboration, and tracking of individual contributions, while automated testing and appropriate integration checks help maintain software quality throughout the system development process. GitHub Copilot continues to support implementation, debugging, and testing activities; however, AI-generated outputs must be reviewed and validated to identify incorrect logic, incomplete requirements, and potential maintainability or security concerns. The project repository on GitHub is available at: https://github.com/SQA-Assignment/Software-Quality-Assurance.

Overall, Assessment 2 focuses on demonstrating measurable progress beyond the initial prototype and evaluating the completed system through implementation evidence, test results, quality metrics, and critical reflection. The final evaluation identifies remaining limitations and provides recommendations for further quality improvement, supporting an evidence-based decision on the system's readiness for release.

2. Project Completion and Final Implementation

2.1. Completed Features

Based on Assessment 1, the Study Room Booking and Management System was further developed to address the limitations of the initial prototype and improve its functionality. The final implementation focused on database integration, administrator functions, GUI improvements, email confirmations, multilingual support, as well as normal and recurring bookings. These features aim to improve booking reliability, usability, and administrative efficiency.

Table below summarises the core features completed in Assessment 2 and the team members responsible for their implementation.

Feature	Status	Completed by
Database	Complete	Phuong
Administrator Functions	Complete	Benjamin
GUI improvements	Complete	Benjamin
Email Confirmation	Complete	Phuong
Multiple Language Support	Complete	Thanh
Normal booking	Complete	Thanh
Recurring booking	Complete	Thanh

Table 1: Completed core features in Assignment 2

2.2. Functional Requirements and Acceptance Criteria

The functional requirements and acceptance criteria from Assessment 1 were reviewed to evaluate the final system implementation. **Table 2** and **Table *3*** summarise the requirements, implementation status, and expected system behaviour, including completed features and those deferred to future development.

No ID	Function Requirement	Status
FR1	Students can search available rooms using date, time, location, capacity and room type filters	Completed in Assignment 1
FR2	Students can book an available room	Completed in Assignment 1
FR3	The system shall validate the availability of study rooms before confirming a booking	Completed in Assignment 1
FR4	The system will validate programme or major eligibility before allowing specialised room bookings	Completed in Assignment 1
FR5	Student will receive booking confirmation via email	Completed in this assignment
FR6	Student can modify future bookings	Completed in Assignment 1
FR7	Student can cancel future bookings	Completed in Assignment 1
FR8	Student can view booking history	Completed in Assignment 1
FR9	Academic staff can create recurring bookings	Completed in this assignment

FR10	Administrators can manage room information and room access rules	Completed in this assignment
FR11	Administrators can override or cancel bookings when necessary	Completed in this assignment
FR12	Administrators can generate room utilisation reports	Pushing this feature to next sprint

Table 2: Completed functional requirements

No ID	Acceptance Criteria	Status
FR1	Given a student is on the search page, When the student enters date, time, location, capacity, and/or room type filters. Then the system shall display only rooms matching all selected criteria (and an appropriate "no rooms found" message if none match).	Complete
FR2	Given a student has selected an available room. When the student submits the booking request. Then the booking will be created successfully, and a confirmation email shall be sent	Complete
FR3	Given a room has already been booked for a selected time. When another user attempts to reserve the same room during the same period. Then the system will reject booking and display an appropriate validation message	Complete
FR4	Given a student attempt to reserve a specialised laboratory. When the student's programme does not satisfy the access requirements. Then the booking shall not be created, and an error message shall be displayed Note: Because the booking results are automatically filtered by role type this is unlikely to happen.	Complete
FR5	Given a booking has been successfully created, When the booking is confirmed. Then the student shall receive a confirmation email containing the room, date, time, and booking reference.	Complete
FR6	Given a student has an existing future booking, When the student modifies the date, time, or room and submits the change. Then the system shall update the booking if the new slot is available or reject the change with a validation message if it conflicts with an existing booking.	Complete
FR7	Given a future booking exists. When the student cancels the booking. Then the booking will be removed, and the room will become available for future bookings	Complete
FR8	Given a student is logged in, When the student opens the booking history page. Then the system shall display all of that student's past and upcoming bookings, including status (completed, cancelled, upcoming).	Complete
FR9	Given academic staff selects the recurring booking option. When they specify a room, time, and frequency. Then the	Complete

	system shall create a linked series of bookings, skipping or flagging any occurrence where the room is already booked.	
FR10	Given an administrator is logged in. When the administrator adds, edits, or removes room information or access rules. Then the system shall save the changes and apply the updated rules to all future bookings.	Complete
FR11	Given an administrator identifies a booking that needs to be cancelled or overridden. When the administrator selects cancel/override on that booking. Then the system cancels the booking, frees the room and notifies the affected student by email.	Complete
FR12	Given an administrator selects a date range. When the administrator generates a room utilisation report. Then the system shall display booking counts and occupancy rates per room for that period.	Pushed to next sprint

Table 3: Completed acceptance criteria

2.3. Changes to Scope, Architecture, and Technology

i. Changes to Scope

The administrator report function was not completed this sprint. It was deferred to a future sprint to prioritise other features and software quality practices in this assessment.

ii. Changes to Architecture (Information Architecture/Design)

The original design and layout were AI generated which had all the features spread out to different pages. This gave the opportunity to make some refinements to better group features together and find more effective ways to layout. These changes were aimed at reduced the number of pages while retaining the required functionality.

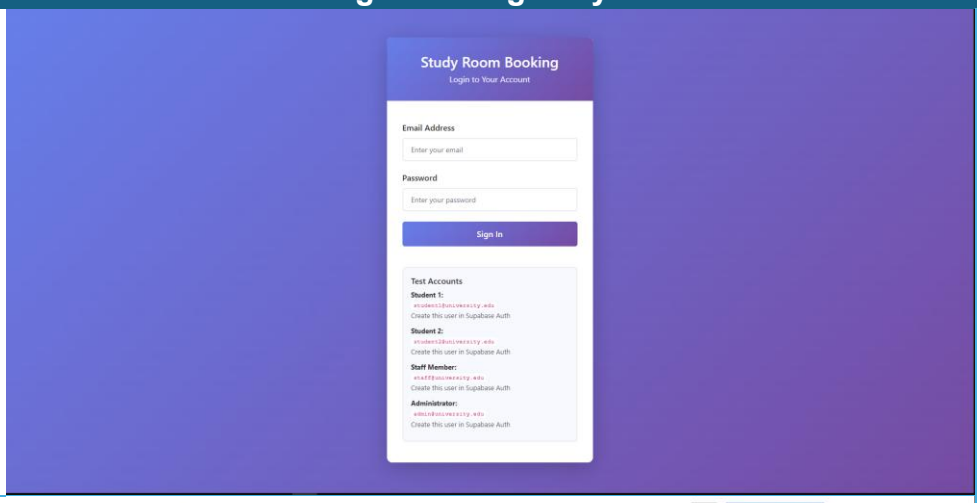
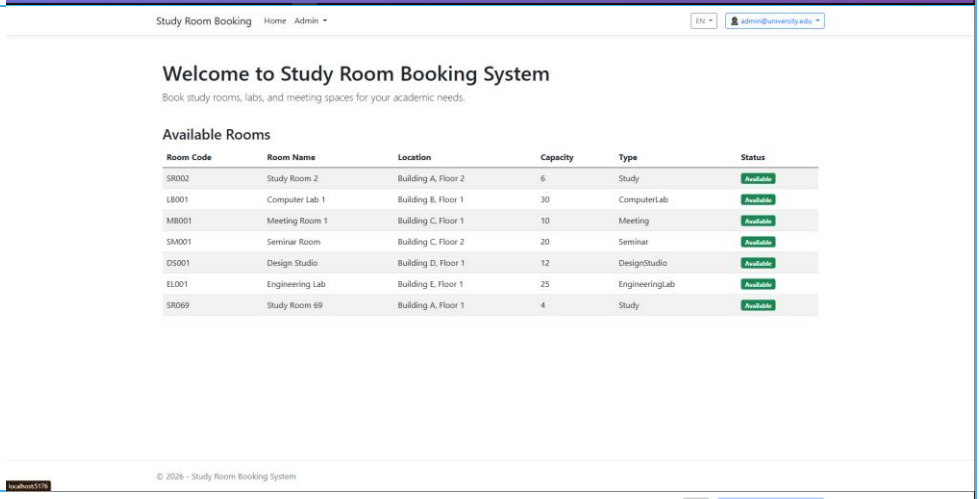
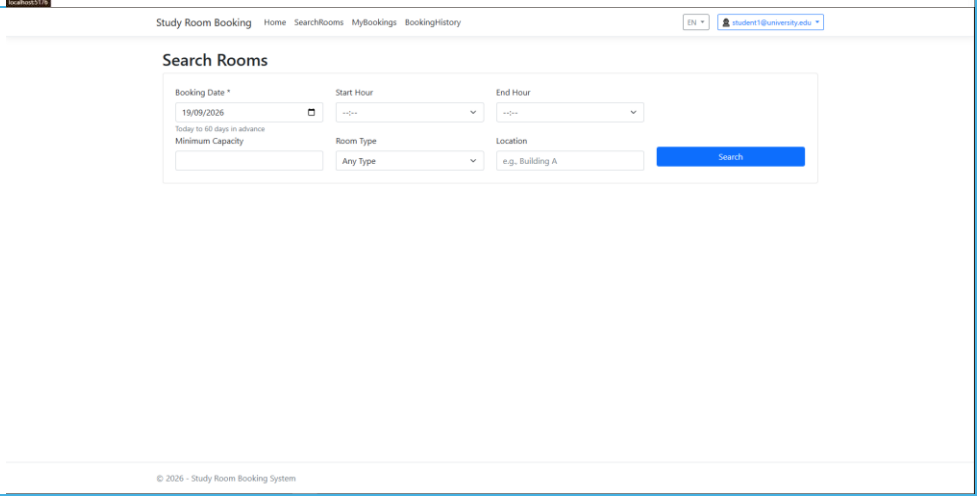
- Bookings and Searching pages combined into one page (see the table below): It is logical that a user would want to search for and book a room at the same time. Having them as separate pages created an unnecessary disconnect between these two parts of the booking process. It also reduces the number of steps required to do a booking as both methods work from one page now.
- My Bookings and Booking History pages combined into one page (see the table below): These are closely related features that make more sense together. This helps consolidate the features once place and reduce number of pages in the navigation bar. Combining these pages makes it easier for users to manage and review their current/past bookings.

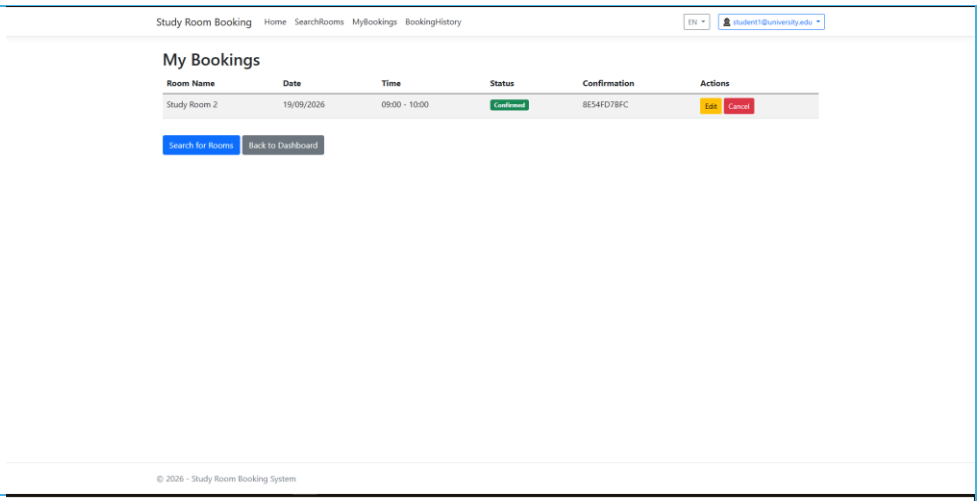
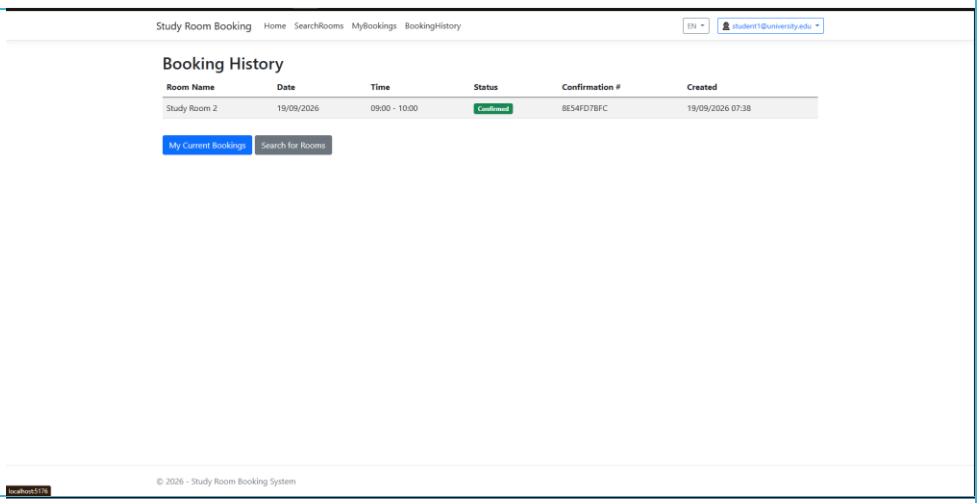
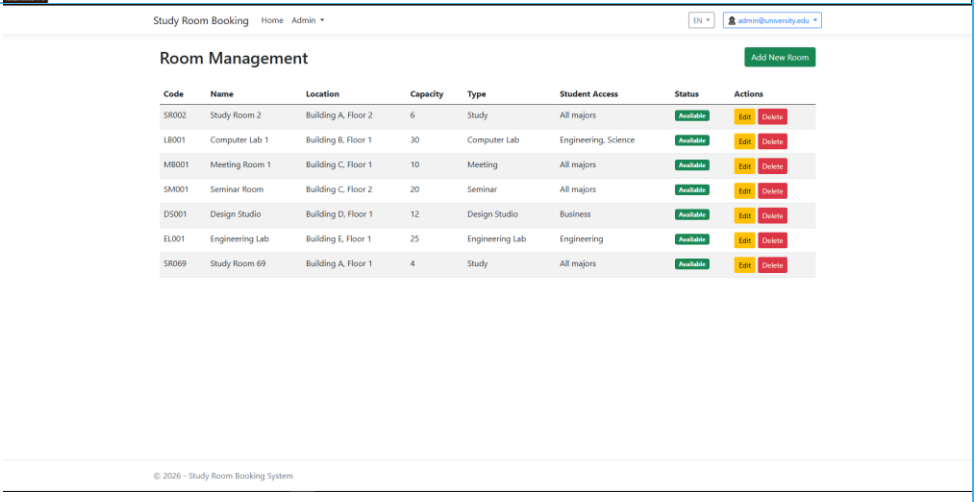
iii. Changes to Technology

The project was changed from using in-memory repositories to database. The change to use database supports the system's non-functional requirements relating to reliability and maintainability. As it provides persistent data storage between sessions and is more appropriate for managing users, rooms, and bookings in the completed system.

iv. Visual Change Log

Design/Layout after completing Assignment 1:

Page Name	Original Design/Layout
Login	
Home (Student/Staff/ Admin)	
Search (Student/Staff)	

My Bookings (Student/Staff)	
Booking History (Student/Staff)	
Room Management (Admin)	

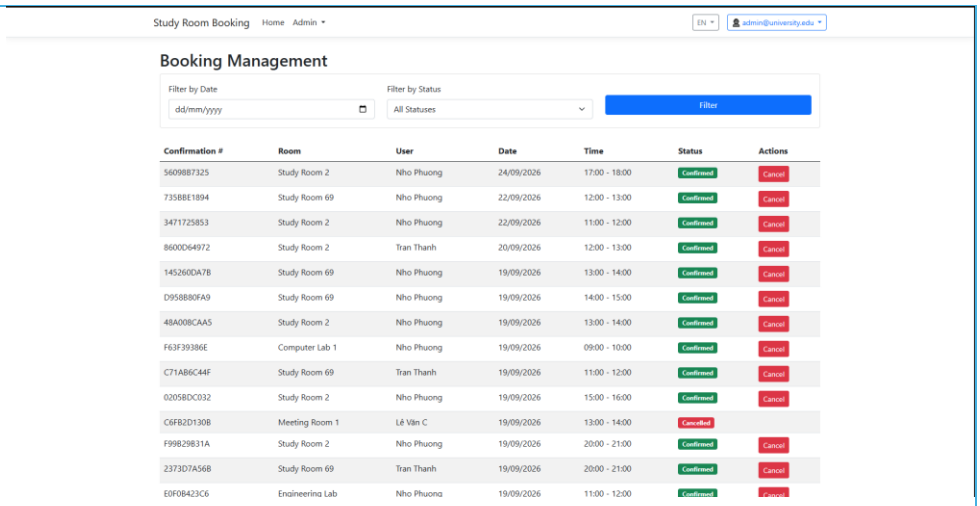
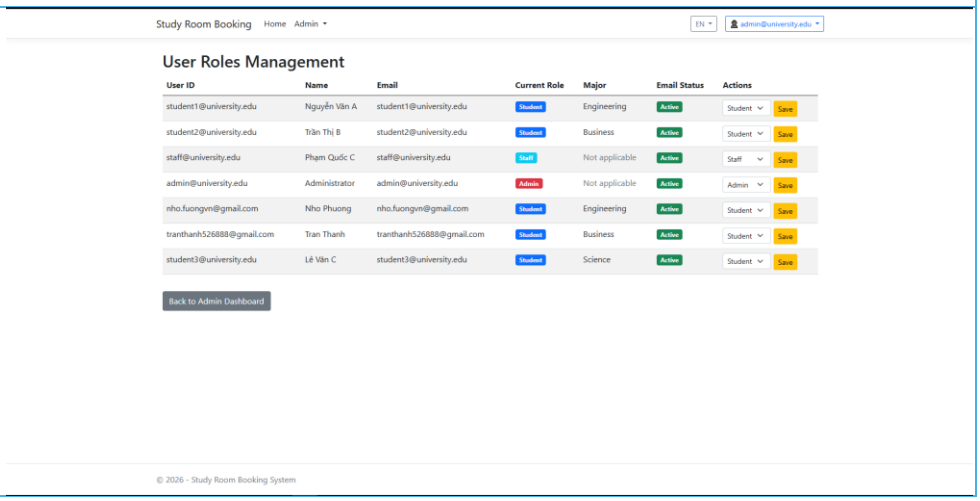
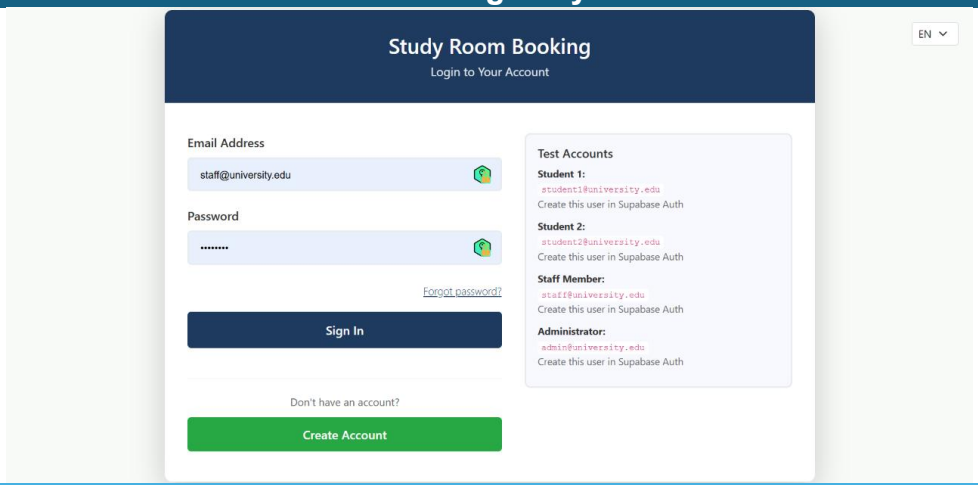
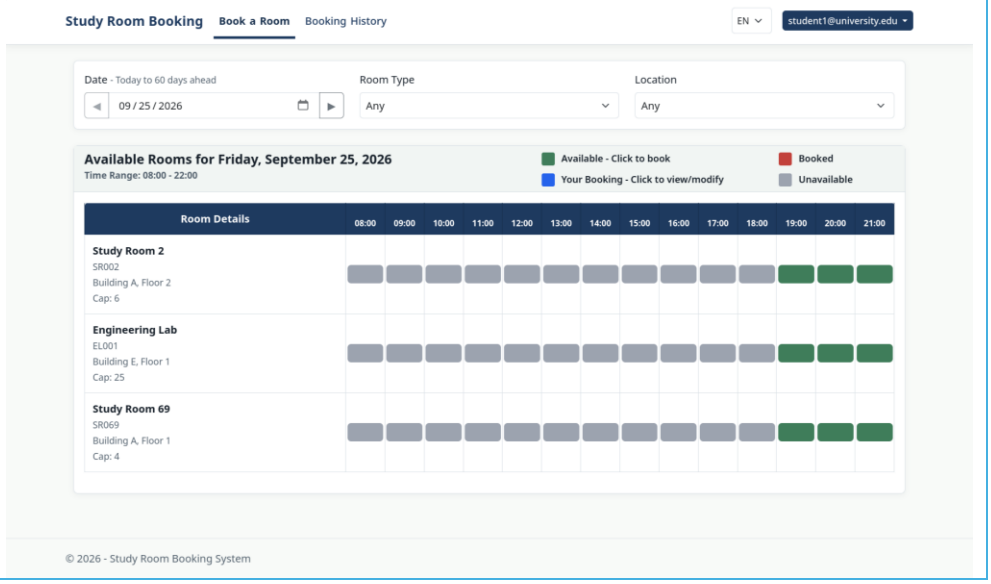
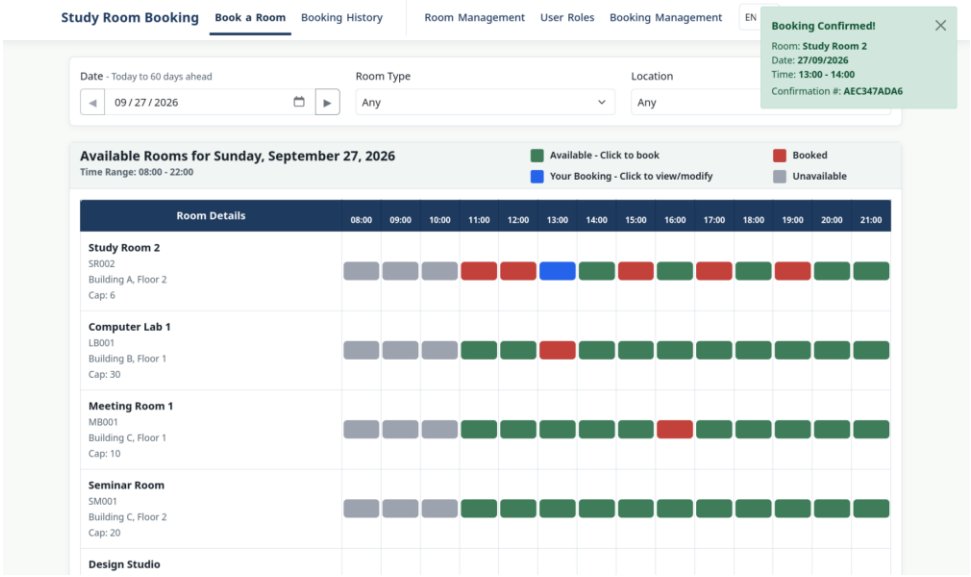
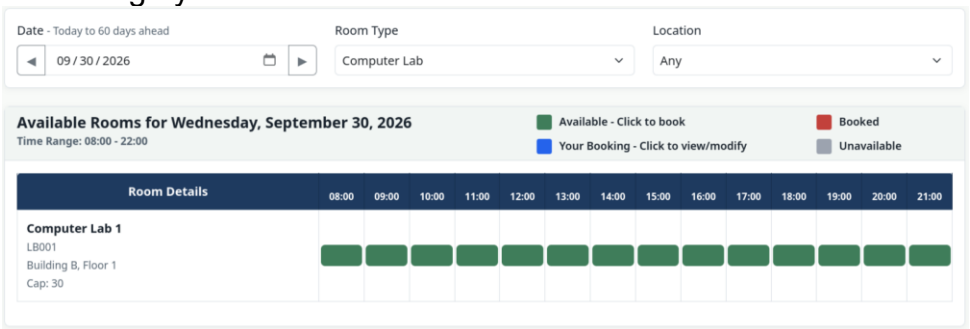
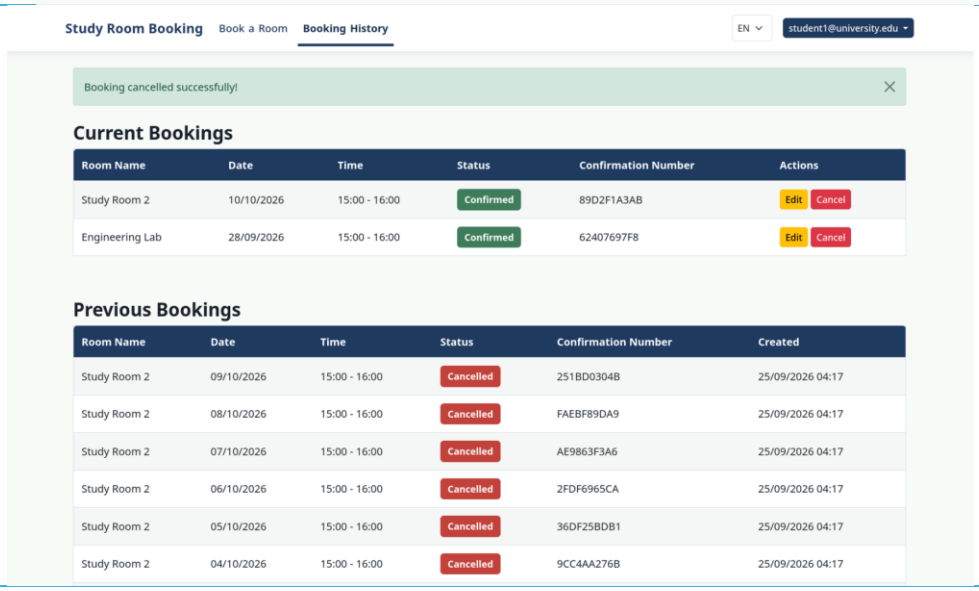
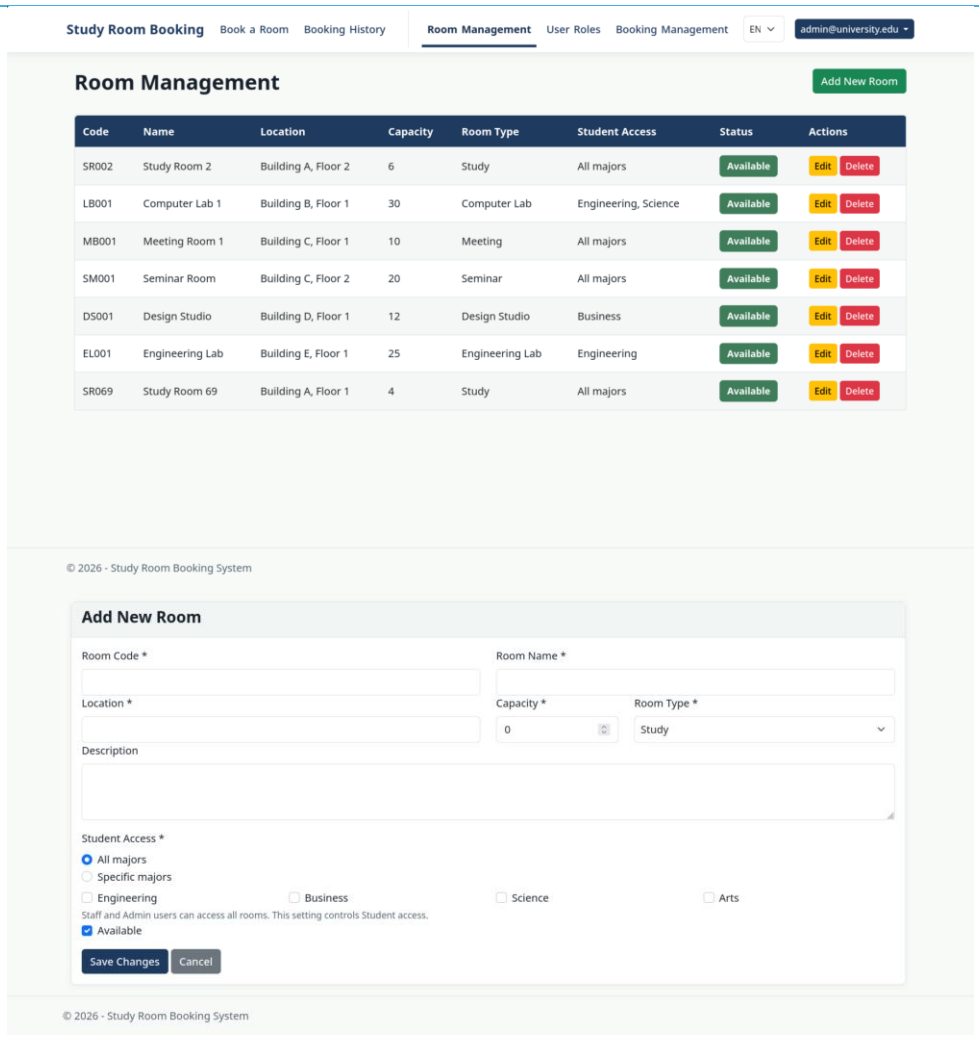
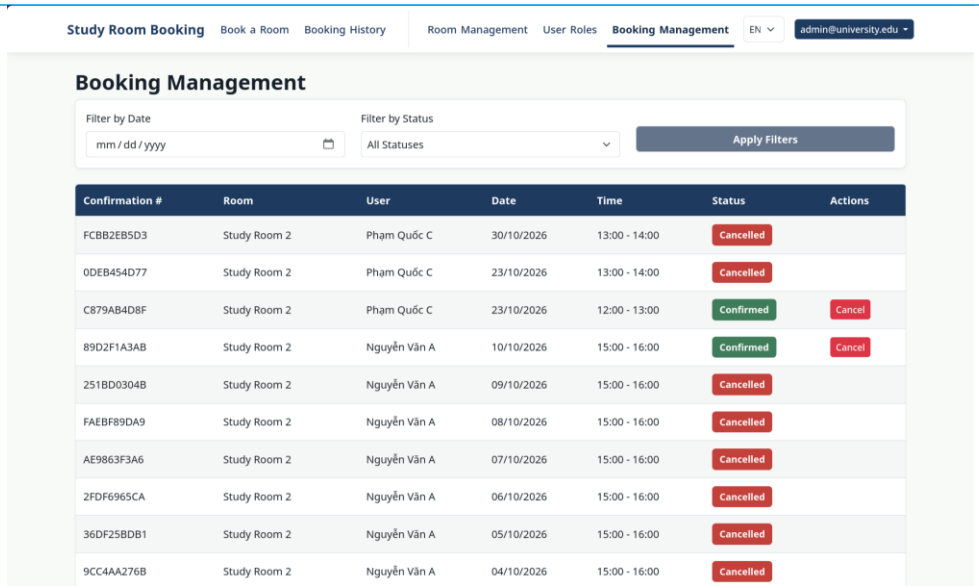
Booking Management (Admin)	
User Roles (Admin)	

Table 4: Design/Layout after completing Assignment 1

Design/Layout after completing this Assignment

Page Name	New Design/Layout
Login	
Home (Student/Staff/ Admin) **+** **Search (Student/Staff)**	• Home 

	• Booking confirmation (see top right)  • Searching by filter 
My Bookings (Student/Staff) **+** **Booking History (Student/Staff)**	

Room Management (Admin)	
Booking Management (Admin)	 Booking management has changed to expired update the status

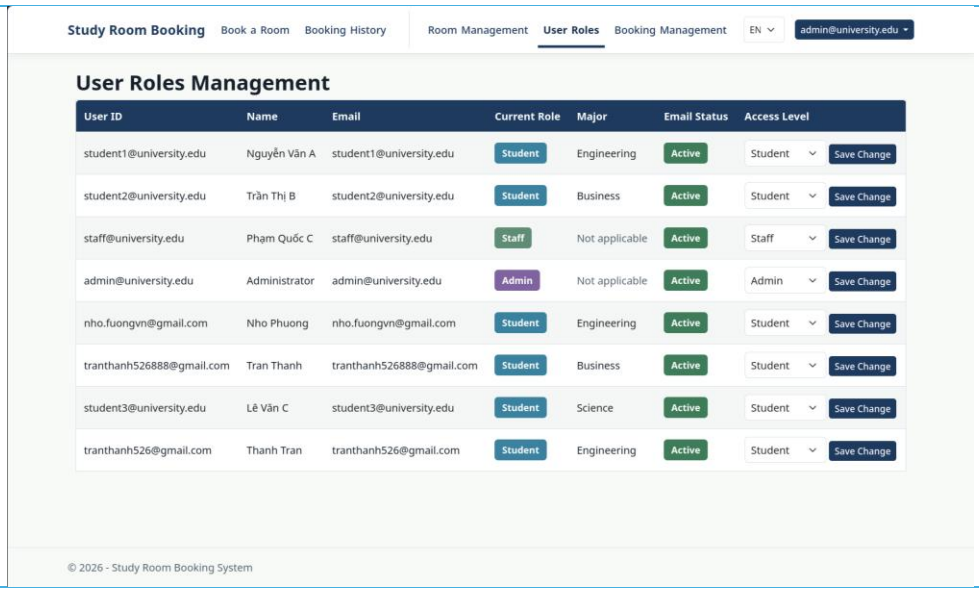
User Roles (Admin)	

Table 5: Design/Layout after completing this Assignment

3. Requirements Verification and Traceability

3.1. Review and Finalisation of Requirements

The functional requirements (FR1 - FR12) and non-functional requirements (NFR1 - NFR7) defined in Assessment 1 have been reviewed against the final implementation. All 12 functional requirements remain valid and relevant to the completed system; no requirements were removed. 11 of the 12 functional requirements have now been implemented. FR12 (administrator reporting) has been formally deferred to a later sprint, as documented in section **2.2**.

No ID	Functional requirements
FR1	Students can search available rooms using date, time, location, capacity and room type filters
FR2	Students can book an available room
FR3	The system shall validate the availability of study rooms before confirming a booking
FR4	The system will validate programme or major eligibility before allowing specialised room bookings
FR5	Student will receive booking confirmation via email
FR6	Student can modify future bookings
FR7	Student can cancel future bookings
FR8	Student can view booking history
FR9	Academic staff can create recurring bookings
FR10	Administrators can manage room information and room access rules
FR11	Administrators can override or cancel bookings when necessary
FR12	Administrators can generate room utilisation reports

Table 6: Functional requirements

No ID	Requirements	Measurement
NFR1	Performance	Room searches will return results within 2 seconds for at least 50 reservations
NFR2	Reliability	The system will prevent duplicate bookings for the same room and time slot
NFR3	Security	Only authorised users may access administrative functions
NFR4	Usability	First-time users will complete a booking within 4 minutes without assistance
NFR5	Maintainability	The system will use a modular architecture to simplify future development
NFR6	Accessibility	Text shall remain readable, and navigation will support keyboard interaction where appropriate. The system will support both Vietnamese and English interfaces, allowing users to switch languages without restarting the application
NFR7	Availability	The prototype shall remain operational throughout demonstration and testing sessions

Table 7: Non-functional requirements

3.2. Updated Requirements Traceability Matrix

The initial 8 test cases (TC-01 to TC-08) designed in Table 8 in Assessment 1 have been reviewed and expanded with 12 additional test cases (TC-09 to TC-20) to achieve broader coverage across all functional and non-functional requirements, bringing the total to 20 test cases ahead of execution in section **4**.

Test ID	Requirement Covered	Test Case Description	Test Steps (Given/When/Then)	Expected Result
TC-01	FR2, FR3	Book an available room successfully	Given a room is available for the selected date/time, when a student submits a booking request, then the booking is created	Booking succeeds BookingResult.Success = true; confirmation message returned
TC-02	FR3, NFR2	Reject a double-booking for the same room and time slot	Given a room is already booked for a time slot, when another student attempts to book the same room and time, then the booking is rejected	BookingResult.Success = false; validation message explains the conflict
TC-03	FR4, NFR3	Hide a specialised room from an ineligible student's search results	Given a student's programme doesn't match a room's access requirement (e.g. a Business student searching while a room is restricted to Design students), when the student searches for available rooms, then the restricted room does not appear in the results	The restricted room is excluded from the returned room list
TC-04	FR4, NFR3	Show a specialised room in an eligible student's search results	Given a student's programme matches a room's access requirement, when the student searches for available rooms, then the restricted room appears in the results	The restricted room is included in the returned room list
TC-05	FR6	Modify an existing future booking to a new available time	Given a student has an existing future booking, when the student changes the booking to a new available date/time, then the booking is updated successfully	BookingResult.Success = true; booking reflects the new date/time, and the original time slot becomes available

TC-06	FR7	Cancel a future booking	Given a future booking exists, when the student cancels the booking, then the booking will be removed, and the room will become available for future bookings	Booking status updated to cancelled; room reappears in availability search
TC-07	FR3 (edge case)	Reject a booking request for a date/time in the past	Given a student attempt to book a room for a date/time that has already passed, when the booking request is submitted, then the system rejects the booking	BookingResult.Success = false; message explains the date is invalid, or ArgumentException thrown
TC-08	FR2 (edge case)	Reject a booking with a missing / invalid student ID	Given a booking request with an empty or null student ID, when the booking is submitted, then the system rejects it	ArgumentException thrown or BookingResult.Success = false with a clear message
TC-09	FR1	Search returns rooms matching all selected filters	Given rooms with varying date, time, capacity, and room type, when a student searches using a combination of filters, then only rooms matching all criteria are returned	Correct filtered result set; no non-matching rooms included
TC-10	FR1 (negative)	Search with no matching rooms returns a clear empty-result message	Given no rooms match the selected filters, when a student searches, then the system displays a “no rooms found” message	Empty result set; clear message shown, no error thrown
TC-11	FR5	Confirmation email is sent with correct booking details	Given a booking is successfully created, when the confirmation email is generated, then the email contains the correct room, date, time, and booking reference	Email content matches the actual booking details
TC-12	FR8	Booking history displays correct status for past and upcoming bookings	Given a student has a mix of expired, cancelled, and upcoming bookings, when the student views booking history, then each booking shows the correct status	All bookings listed with accurate status labels

TC-13	FR9	Recurring booking creates a linked series, skipping conflicts	Given academic staff request a weekly recurring booking over 4 weeks, when one occurrence conflicts with an existing booking, then the system creates the other 3 bookings and flags/skips the conflicting one	3 bookings created; conflicting occurrence flagged, not silently dropped
TC-14	FR10, NFR3	Administrator can update room access rules	Given an administrator is logged in, when they update a room's access rules (e.g. restrict to Design students), then the new rule applies to all future bookings of that room	Updated rule saved and enforced on subsequent booking attempts
TC-15	FR11, NFR3	Non-administrator cannot override or cancel another user's booking	Given a logged-in student (non-admin), when they attempt to cancel another student's booking, then the system denies the action	Action blocked; appropriate authorisation error returned
TC-16	FR11	Administrator override cancels booking and notifies the affected student	Given an administrator overrides an existing booking, when the override is confirmed, then the booking is cancelled, the room becomes available, and the student is notified	Booking cancelled; room freed; notification sent/logged
TC-17	NFR1 (Performance)	Room search returns results within 2 seconds for 50+ reservations	Given at least 50 sample reservations exist in the database, when a student performs a search, then results are returned within 2 seconds	Search completes within the 2-second threshold
TC-18	NFR4 (Usability)	First-time user completes a booking within 4 minutes unassisted	Given a first-time user with no prior instruction, when they attempt to search for and book a room, then they complete the booking within 4 minutes	Booking completed within the time threshold, without external help
TC-19	NFR6 (Accessibility)	Language switch applies	Given a user is on any page of the application,	All visible text updates to the selected

		immediately without restarting the application	when they select a different language (Vietnamese/English), then the interface text updates without requiring the program restart	language; no restart required
TC-20	NFR2, FR3 (Regression)	Re-verify double-booking prevention after database migration	Given the system now uses a database instead of in-memory storage, when two students attempt to book the same room and time concurrently, then the system still correctly rejects the conflicting booking	Double-booking still prevented after the architecture change (regression check)

Table 8: Test cases and expected results

Requirement ID	Requirement Summary	Related NFR (if any)	Test Cases	Status
FR1	Search available rooms by filters	NFR1	TC-09, TC-10	Passed
FR2	Book an available room		TC-01, TC-08	Passed
FR3	Validate room availability before confirming booking	NFR2	TC-01, TC-02, TC-07, TC-20	Passed
FR4	Validate programme/major eligibility for specialised rooms	NFR3	TC-03, TC-04	Passed
FR5	Send booking confirmation via email		TC-11	Passed
FR6	Modify future bookings		TC-05	Passed
FR7	Cancel future bookings		TC-06	Passed
FR8	View booking history		TC-12	Passed
FR9	Academic staff can create recurring bookings		TC-13	Passed
FR10	Administrators manage room information and access rules	NFR3	TC-14	Passed
FR11	Administrators override or cancel bookings	NFR3	TC-15, TC-16	Passed
FR12	Administrators generate room utilisation reports		None-out of scope	Deferred (see 2.2)

Table 9: Functional requirements traceability matrix

Requirement ID	Requirement Summary	Test Cases	Status
NFR1	Performance (search ≤2s, 50+ reservations)	TC-17 + manual quality testing in section 8	Passed
NFR2	Reliability (no duplicate bookings)	TC-02, TC-20	Passed
NFR3	Security (authorised access only)	TC-03, TC-04, TC-14, TC-15, TC-16 + manual quality testing in section 8	Passed
NFR4	Usability (4-min unassisted booking)	TC-18	Passed
NFR5	Maintainability (modular architecture)		Supported by database migration and CI/CD, no direct test case
NFR6	Accessibility (language switch; keyboard nav)	TC-19	Partially covered – TC-19 verifies language switching only
NFR7	Availability (operational during demo/testing)		Observed during testing sessions, not a discrete test case

Table 10: Non-functional requirements traceability matrix

3.3. Requirements Not Fully Verified or Out of Scope

- **FR12 - Administrators can generate room utilisation reports**
  Formally deferred to a later sprint (see section **2.2**). The team prioritised database implementation, administrator management, email confirmation, multi-language support, and recurring bookings for this assignment. FR12 remains valid and will be addressed in a future iteration.
- **NFR6 - Partially covered**
  The original test case for NFR6 covered both keyboard navigation and language switching. Following a revision to TC-19, the test case now verifies language switching only; keyboard navigation through the booking flow is not currently covered by any test case. This is a known gap the team has chosen to accept at this stage, given the project's limited timeframe and focus on core booking functionality; it is flagged here for transparency rather than silently omitted and may be addressed in a future iteration if time permits.
- **NFR5 and NFR7 - No discrete test case**
  NFR5 (Maintainability) is supported by the architectural decision to migrate from in-memory repositories to a database (section **2.2**). Rather than verified through a specific automated test. NFR7 (Availability) will be confirmed through observation during demonstration and testing sessions rather than a standalone test case, as uptime is not something a unit or integration test can meaningfully simulate for this prototype.

4. Test Execution and Final Test Results

4.1. Test Execution

The final system was tested using 20 test cases with combination of automated unit tests and manual system testing. The automated tests were implemented using MSTest and an isolated in-memory SQLite database to verify booking, availability, access control, cancellation, booking history, and performance-related behaviour without affecting the application's actual database. Manual testing was used for functions that depend on the complete web application, including confirmation email delivery, recurring bookings, administrator functions, usability, language switching, and database persistence.

Both positive and negative scenarios were included. These covered successful bookings, double-booking prevention, restricted-room access, invalid dates, missing user information, unauthorised actions, performance, and other system-level behaviours.

4.2. Final Test Cases and Execution Results

Test ID	Test Type	Requirement	Test Scenario	Execution Method	Result
TC-01	Unit / Functional	FR2, FR3	Book an available room successfully	Automated	**Pass**
TC-02	Unit / Negative	FR3, NFR2	Reject an overlapping double booking	Automated	**Pass**
TC-03	Unit / Security	FR4, NFR3	Hide restricted room from ineligible student	Automated	**Pass**
TC-04	Unit / Functional	FR4, NFR3	Show restricted room to eligible student	Automated	**Pass**
TC-05	Unit / Functional	FR6	Modify a future booking to an available time	Automated	**Pass**
TC-06	Unit / Functional	FR7	Cancel a future booking	Automated	**Pass**
TC-07	Unit / Negative	FR3	Reject a booking for a past date	Automated	**Pass**
TC-08	Unit / Invalid Input	FR2	Reject booking with missing/invalid student ID	Automated	**Pass**
TC-09	Functional	FR1	Search using all selected room filters	Manual	**Pass**
TC-10	Negative / Boundary	FR1	Search with criteria that match no rooms	Manual	**Pass**
TC-11	Functional	FR5	Send confirmation notification with correct booking details	Manual	**Pass**
TC-12	Unit / Functional	FR8	Display correct booking history and statuses	Automated	**Pass**

TC-13	System / Functional	FR9	Create recurring weekly bookings and handle a conflict	Manual	**Pass**
TC-14	Unit / Security	FR10, NFR3	Verify room access rules for different student majors	Automated	**Pass**
TC-15	Security / Negative	FR11, NFR3	Prevent non-admin user cancelling another user's booking	Automated	**Pass**
TC-16	System / Security	FR11, NFR3	Administrator overrides and cancels another booking	Manual	**Pass**
TC-17	Performance	NFR1	Search with 60 rooms and 300 bookings within 2 seconds	Automated	**Pass**
TC-18	Usability / Acceptance	NFR4	First-time user completes a booking within 4 minutes	Manual	**Pass**
TC-19	Accessibility / Language	NFR6	Switch between Vietnamese and English without restarting the application	Manual	**Pass**
TC-20	Integration / Regression	NFR2, FR3	Re-verify double-booking prevention after database migration	Manual	**Pass**

Table 11: Final test cases and execution results

i. Automated Test Results

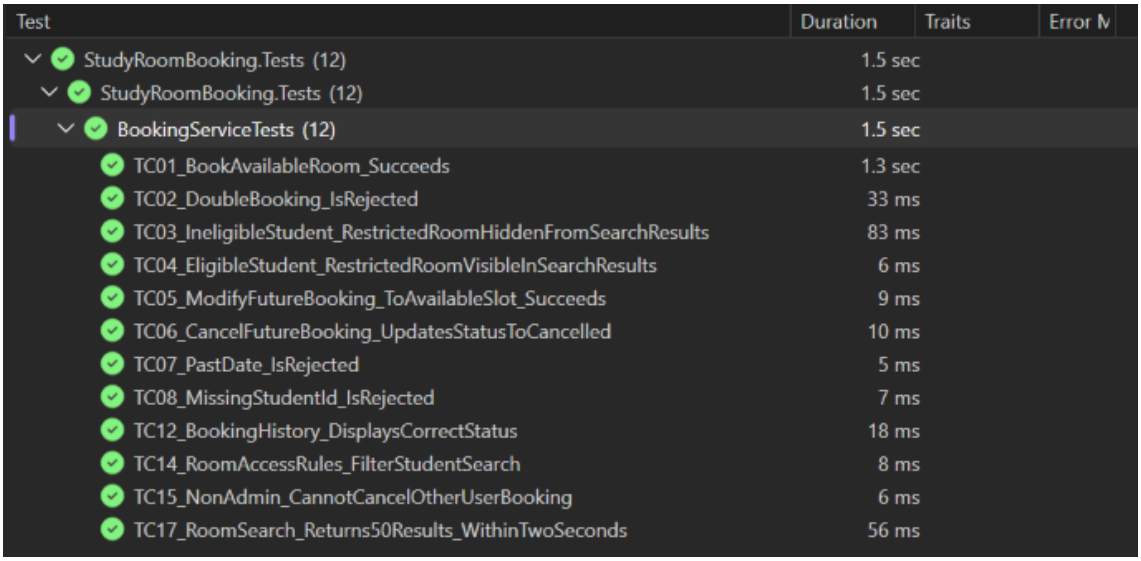


Figure 1: Automated test results

ii. Manual Test Results

- **TC-09: Search using all selected room filters**

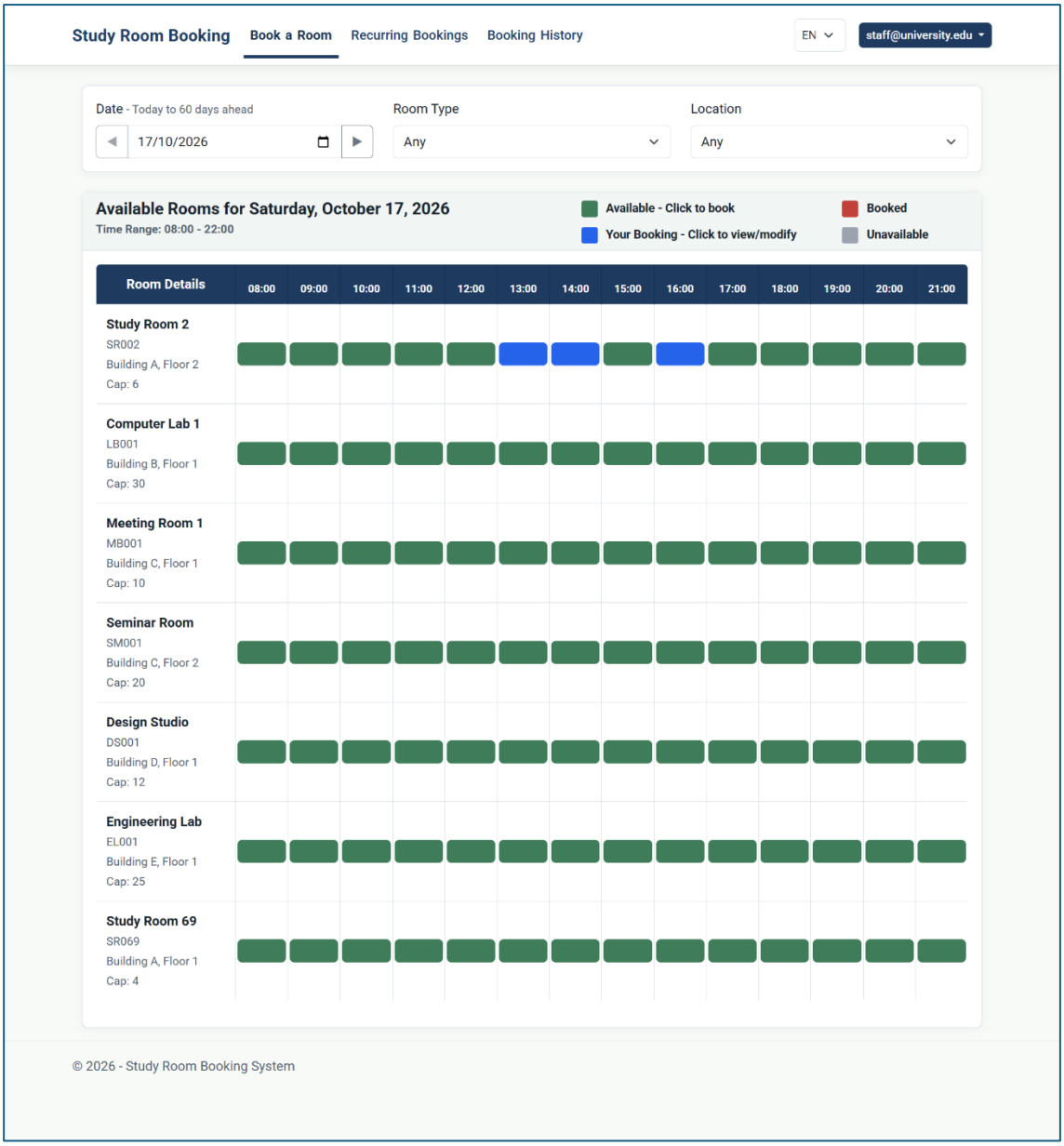


Figure 2: Search using all selected room filters

- **TC-10: Search with criteria that match no rooms**

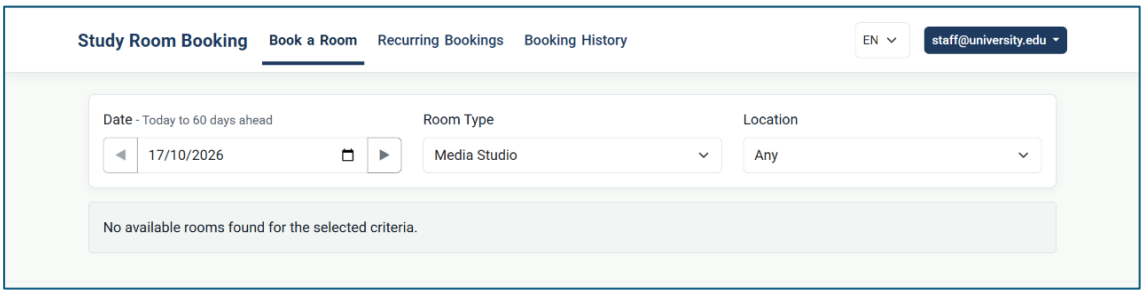


Figure 3: Search with criteria that match no rooms

- **TC-11: Send confirmation notification with correct booking details**

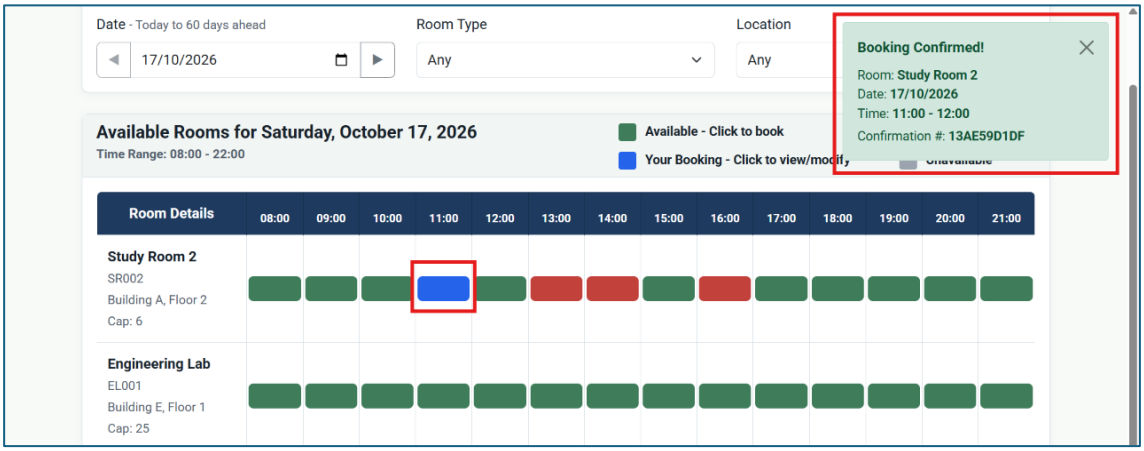


Figure 4: Booking a slot

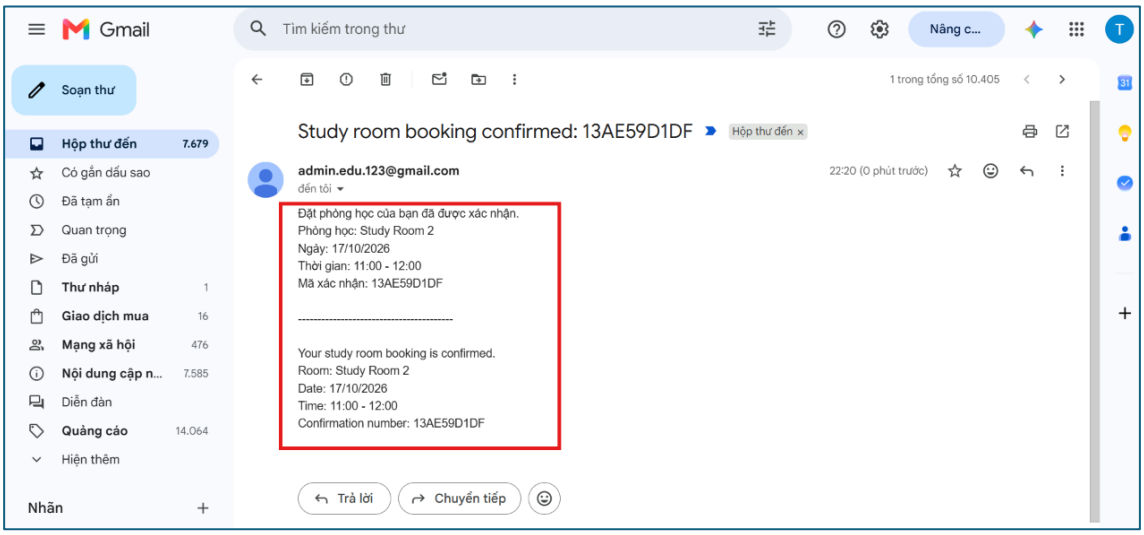


Figure 5: Send confirmation notification with correct booking details

- **TC-13: Create recurring weekly bookings and handle a conflict**

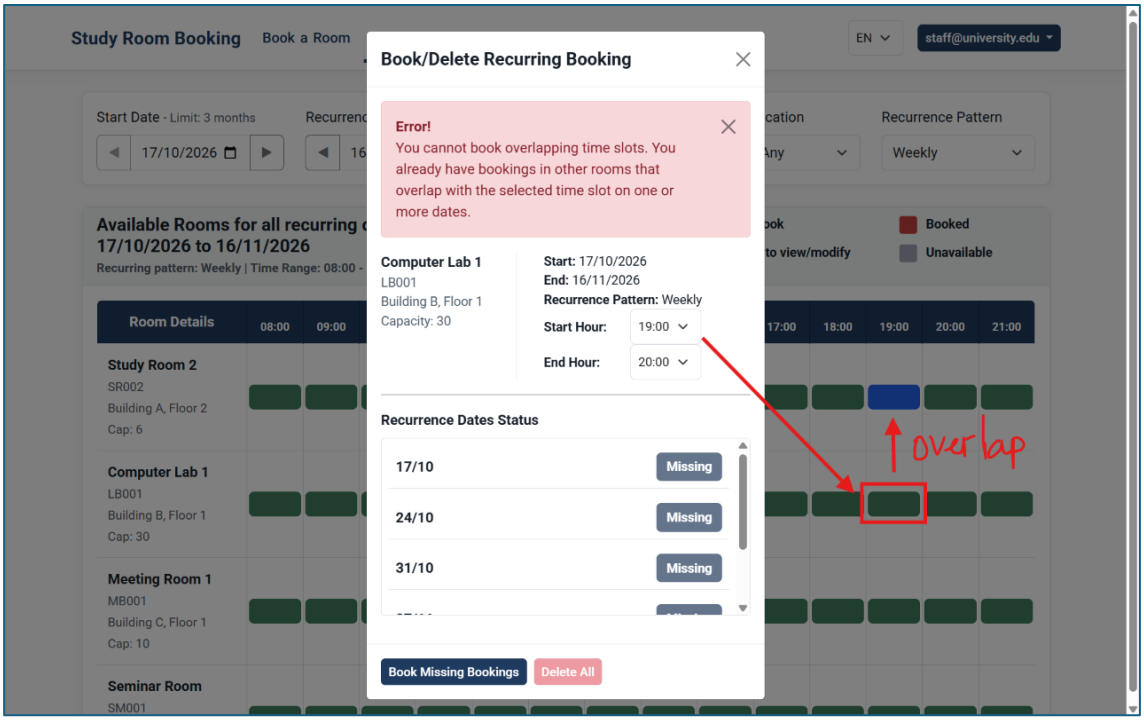


Figure 6: Create recurring weekly bookings and handle a conflict

- **TC-16: Administrator overrides and cancels another booking**

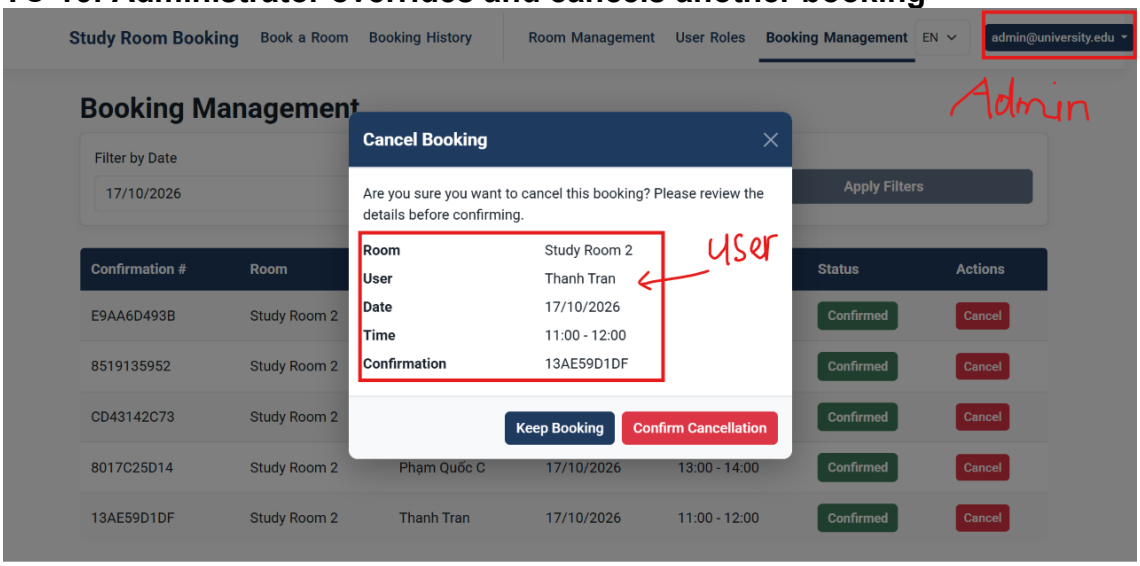


Figure 7: Administrator cancels a user's booking

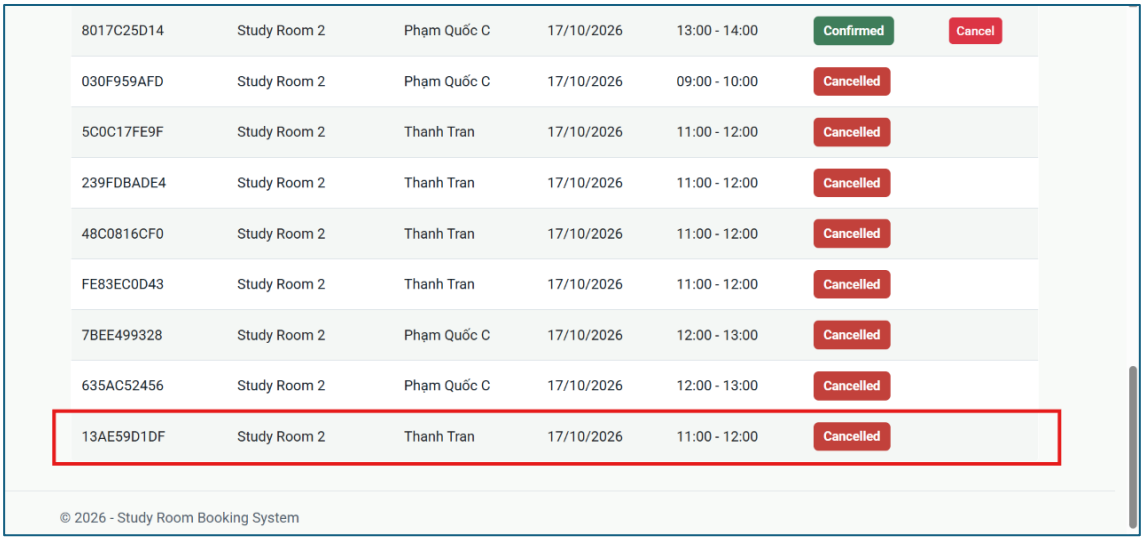


Figure 8: Booking cancelled and updated status

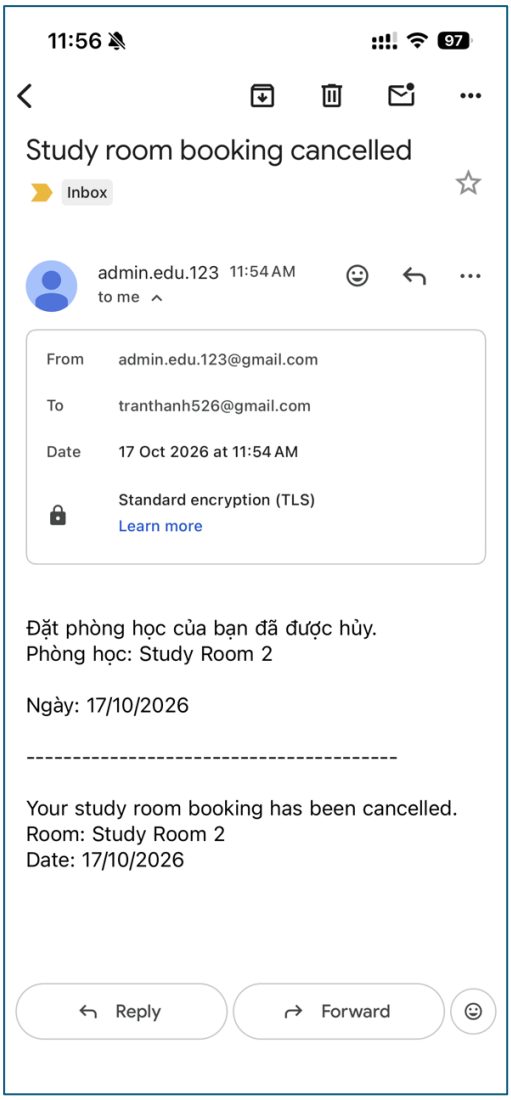


Figure 9: Cancellation email sent to the user

- **TC-18: First-time user completes a booking within 4 minutes**

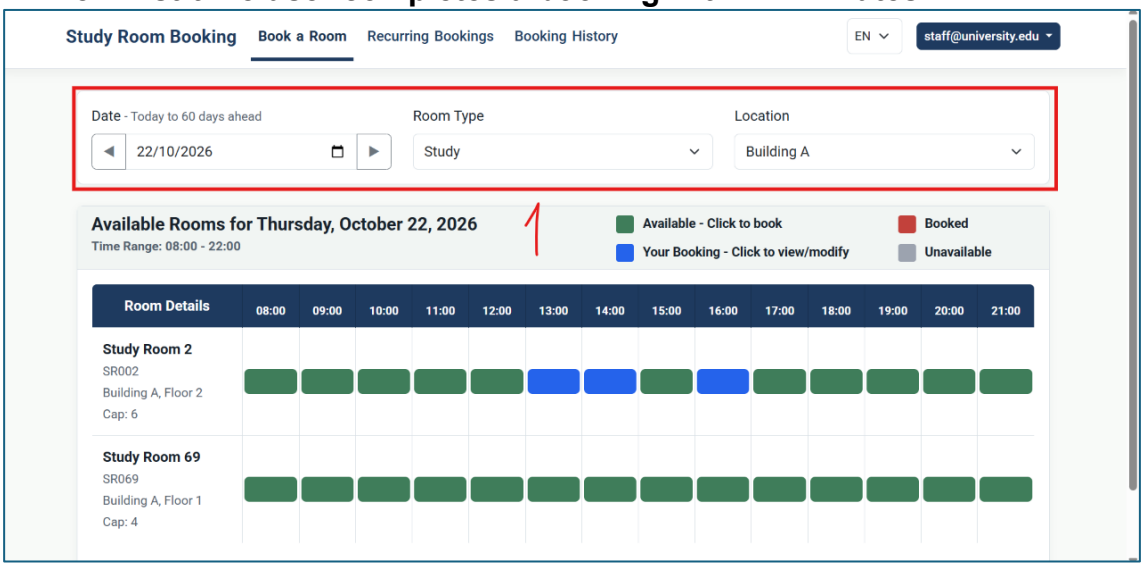


Figure 10: Booking's step 1 - select search criteria

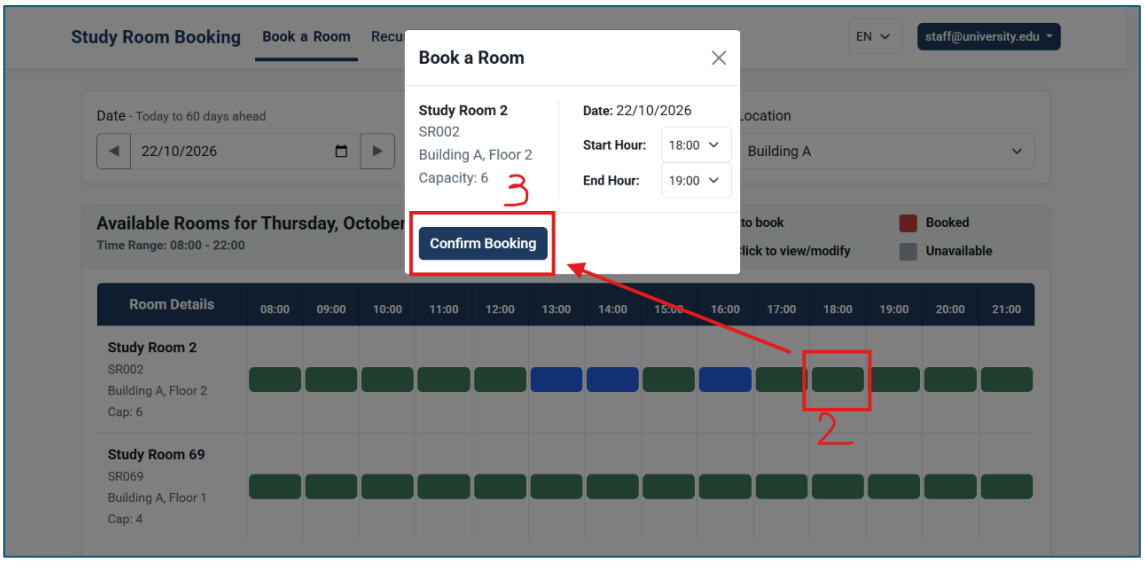


Figure 101: Booking's step 2 and 3 - select slot and confirm booking

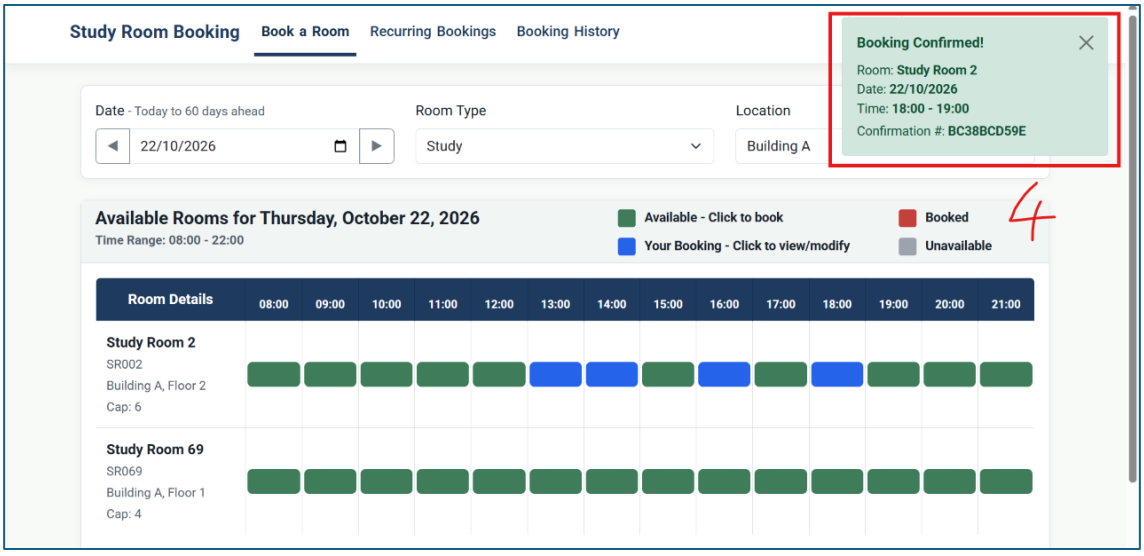


Figure 11: Booking's step 4 - view booking success details

- **TC-19: Switch between Vietnamese and English without restarting the application**

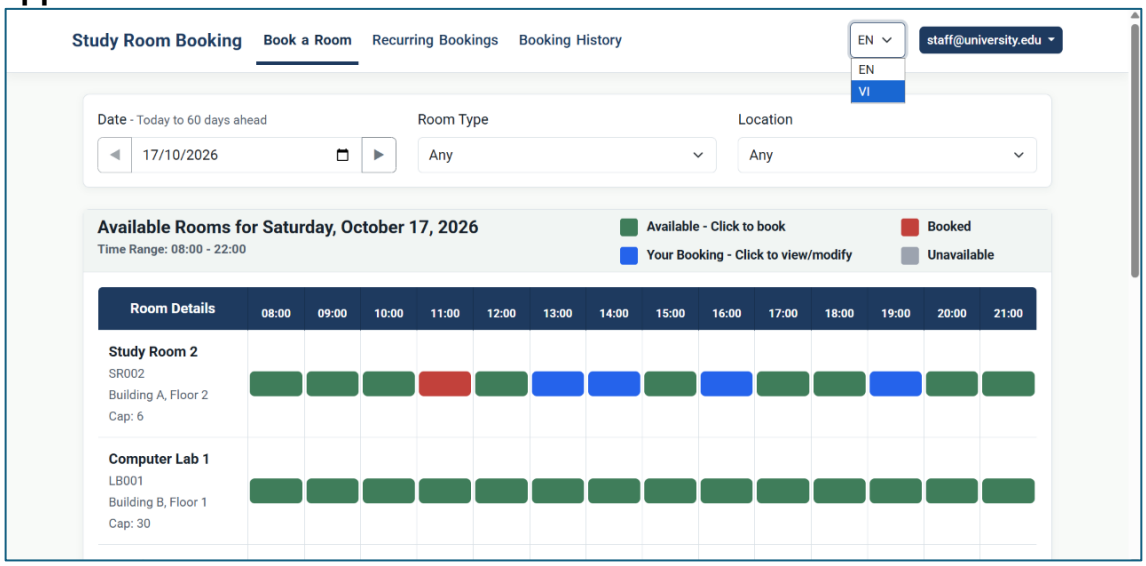


Figure 12: Web UI in English

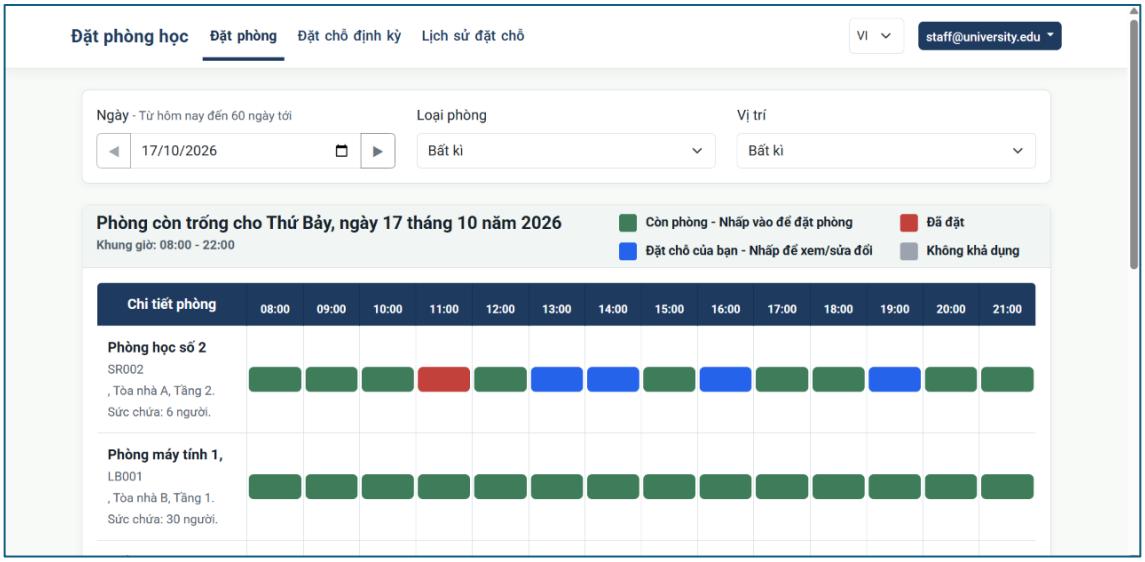


Figure 14: Web UI after switching to Vietnamese

- **TC-20: Re-verify double-booking prevention after database migration**

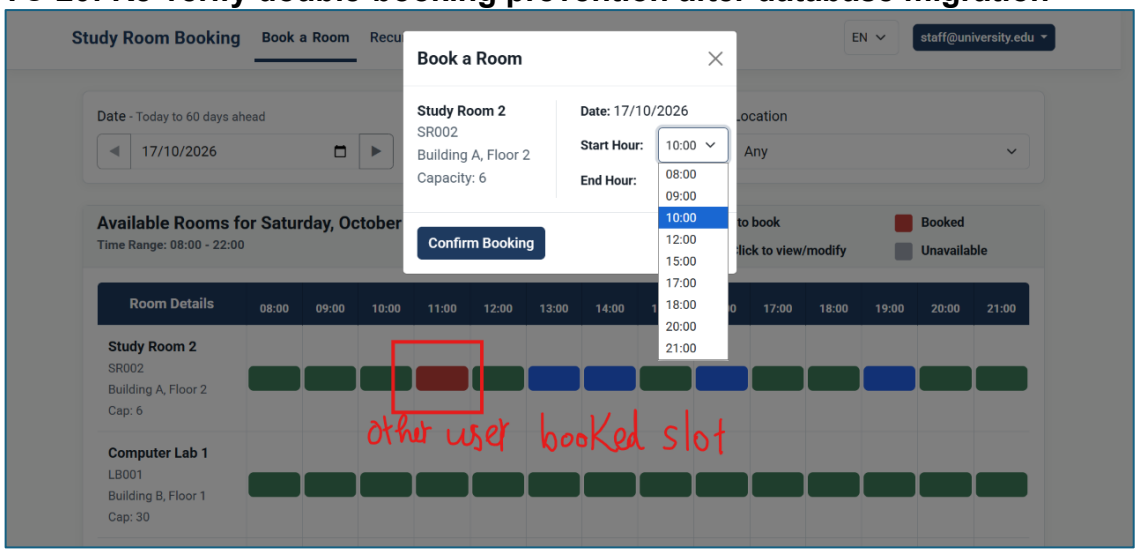


Figure 13: Start hour options excluding hours already booked by another user

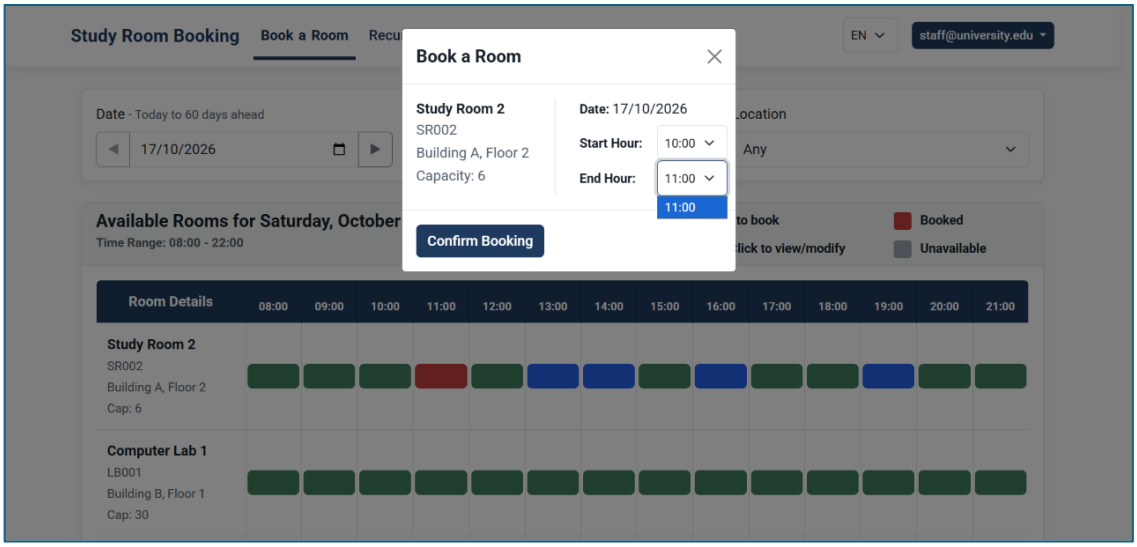


Figure 14: End hour options excluding hours already booked by another user

4.3. Test Execution Summary

The final test execution consisted of 20 planned test cases. The automated tests were executed using the MSTest test project, while the remaining system-level tests were performed manually through the completed web application.

Test Result	Number
Planned	20
Executed	20
Passed	20
Failed	0
Blocked	0
Not Run	0

Table 12: Test execution summary

The automated test suite covered the main booking-service functions and produced repeatable results. Manual testing provided additional coverage of complete user workflows and features that could not be adequately verified using mocked repositories alone.

4.4. Failed Tests and Corrective Actions

All 20 planned test cases passed during final execution, so no failed tests required corrective action. The automated tests confirmed that the main booking functions continued to operate correctly after the system changes.

During development, issues identified through testing were corrected before the final execution. These included validation of booking information, room access restrictions, cancellation behaviour, and integration with the database. The affected tests were then rerun to confirm that the corrections did not introduce new problems.

4.5. Regression Testing

Regression testing was performed after implementing the database and additional booking features. The original booking tests from Assignment 1 were rerun together with the new tests to ensure that existing functionality was not affected by the changes.

Tests such as TC-01, TC-02, TC-05, TC-06, and TC-07 confirmed that creating, preventing conflicting, modifying, cancelling, and validating bookings continued to work correctly. TC-20 additionally re-verified that double-booking prevention continued to work after the database migration.

The regression testing resulted in all relevant tests passing, indicating that the new implementation did not introduce failures into the previously tested booking functionality.

4.6. Remaining Quality Risks

Although all final test cases passed, several quality risks remain:

- FR12: Room utilisation reporting was deferred and was not included in the final implementation.
- NFR1: The performance test used an isolated in-memory SQLite database, so it does not fully represent performance with the application's actual database in a deployed environment.
- Email delivery: Automated testing verified that the notification service was called with the correct information. Actual external email delivery depends on the configured email service.
- Security: The tests verified important application-level access restrictions, but a full security assessment was outside the scope of this project.

Overall, the final testing achieved 20 passed out of 20 executed tests. The results provide evidence that the implemented booking, access control, notification, database, usability, performance, and regression functions operated as expected during final testing. The remaining risks provide areas for future improvement and further testing.

5. Test Automation and Quality Engineering

5.1. Test Case Selection for Automation

Test cases for automation were identified based on whether the behaviour of the code was repeatable, had clearly defined expected results and could be reliably verified using test data. Tests that required visual confirmation or interaction with the user interface, such as checking whether the language switcher worked correctly, were left to manual testing.

The MSTest framework was used because it integrates well with the .NET development environment and allows to run test cases automatically. For these test cases rather than using Moq repositories to simulate database behaviour, we have moved to using an isolated in-memory SQLite relational database. This allows the tests to interact with the database layer while keeping each test independent from the application's normal database and from other tests.

A limitation of this approach is that it does not completely replicate the actual database in use. However, it allows testing features involving a database without risking the modification of any real data.

5.2. Implementation of Automated Tests

The automated test cases were implemented using the MSTest framework and using an isolated SQLite database. Each test creates a fresh database during setup and disposes of it after the test has finished. This tests data a more realistic way as the data will be retrieved from a simulated database. This approach protects the actual database from being modified while doing a better job at creating a realistic test environment.

5.3. Automated Test Execution Results

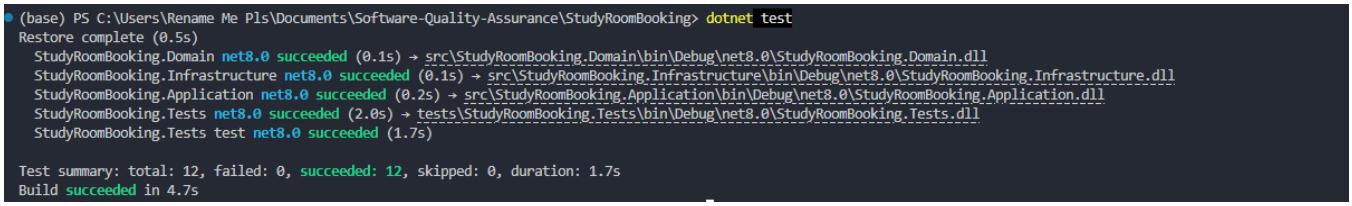


Figure 15: Automated test execution results

All automated tests passed when executed, with no test failures or errors.

5.4. Benefits, Limitations, and Maintainability

The main benefit of having the automated test suite is that it allows source code to be tested quickly and repeatedly without relying entirely on manual testing. This is particularly useful when implementing regression testing after the implementation of the database. As this helps determine whether using the database broke any logic or introduced any defects in the code.

Another benefit is the ability to simulate the functionality of database for testing, without affecting the real one. This protects real booking data from being modified or corrupted while running the tests. Each test also has its own isolated database and test data, making the results more consistent and reducing the chance of other tests affecting each other.

A limitation of this is that the in-memory SQLite database used for testing is not the same as the actual database used by the application. Therefore, it may not identify issues that are specific to the actual database system. However, it still allows the application's database interactions and code functionality to be tested without risking real booking data.

The test suite is maintainable because the test data is created within the tests and each test starts with an isolated database. If application requirements or expected test behaviour changes and regression testing is needed, the relevant test cases can easily be updated and rerun to check that the changes have not introduced any new defects.

5.5. Justification for Manual and Exploratory Testing

Manual testing was used for test that involved visual confirmation to ensure that they are meeting their requirements. As job testing something that requires human observation and understanding is too difficult to test automatically.

For the translation feature (TC-19) manual testing was needed because it required people who understood the language to review the translations. This helped identify an issue with the wording on the booking page. On this page, the room-time matrix displays the available rooms, and the word "Free" was originally used to indicate that a room was not currently in use. However, when translated into Vietnamese, the wording could be interpreted as meaning that the room is free of charge. Because the rooms were already free to use, this could have been confusing to users. Manually testing allowed us to identify this issue and change the wording to make its meaning clearer.

Another example was the room search and filtering functionality. While automated tests could verify that the correct results were returned, manual testing was useful for checking that the filters were understandable and that the results were presented clearly to the user. Different combinations of filters were tested manually, including searches where no rooms matched the selected criteria (TC-09 and TC-10).

Manual testing was also used for functionality where the complete user experience or an external service needed to be checked. This included recurring bookings (TC-13), administrator actions (TC-16), usability testing for first-time users (TC-18), and checking that confirmation emails were actually received (TC-11). The automated TC-11 test could verify that the notification service was given the correct booking details, but manual testing was still required to confirm that the email went into that person’s inbox.

6. Continuous Integration and DevOps QA

6.1. Development and Integration Workflow

The team uses a feature-branch workflow on GitHub after learning about the CI/CD lecture. From now on, each member develops on a separate branch and commits small, clearly described changes. When a change is ready, the member opens a pull request into main. The main branch is protected (Code Review): direct pushes are blocked, and a pull request can only be merged when the automated checks pass and at least one teammate has approved it.

The flow is:

1. Create a branch from main for a feature or fix
2. Commit locally and run dotnet build and dotnet test before pushing
3. Push the branch and open a pull request
4. GitHub Actions runs the CI workflow automatically
5. A teammate reviews the code and the CI result, then approves or requests changes
6. After approval and a green build, the pull request is merged into main. CI runs again on the merged code

CI is appropriate because the project is a small .NET application with an automated MSTest suite and a shared repository, so frequent integration reduces merge problems and catches faults early. Continuous deployment was not implemented because the prototype is only run in demonstration and testing sessions, so there is no production environment to deploy.

6.2. CI Implementation

The workflow is defined in .github/workflows/ci.yml. It runs on every push and pull request to main and uses read-only repository permissions. It has two jobs.

- Job 1: Restore, build and test. This job checks out the code, installs .NET 8, restores packages, builds the solution in Release mode, runs a basic lint check, runs all automated tests with code coverage, and checks the coverage against a minimum. Test results and coverage reports are saved as an artefact after every run.
- Job 2: Smoke test. This job only runs if Job 1 passes. It starts the web application and sends HTTP requests until the app responds, for up to about 90 seconds. The application connects to the database using a connection string stored as a GitHub repository secret, so no credentials appear in the code or logs. If the secret is missing or the app does not respond, the job fails and prints the application log.

6.3. Quality Gates

Gate	Check	Purpose	Related Requirement
1. Build	Solution restores and compiles in Release mode	Prevents broken code reaching main	All
2. Linting	Dotnet format analysers report no errors	Catches code quality problems automatically	NFR5 Maintainability

3. Automated tests	All 20 tests pass (12 automated tests and 08 manual tests)	Protects rules such as duplicate-booking prevention	FR2, FR3, NFR2
4. Coverage threshold	Line coverage of at least 30%	Ensures a minimum amount of code is tested	NFR2, NFR5
5. Smoke test	App starts and responds with the real database connection	Confirms the system runs, not only complies	NFR7 Availability
6. Code Review	At least one approving review before merge (Code Review)	Catches logic and design problems automation cannot	All

Table 13: Quality gates table

If any gate fails, the pull request cannot be merged until the problem is fixed. The coverage threshold is low (30%) because the project is a prototype. The team plans to raise it as more tests are added. The lint check only reports analyser errors, so it is a basic check and does not enforce full code style.

6.4. Smoke and Sanity Testing

- **Smoke testing:** The smoke test is automated in CI. It does not test business rules. It checks only that the application starts, connects to its configuration, and responds. HTTP 200 means the page loaded, and 302 means the app redirected the request, for example, to a login page. Both show the app is running. This catches faults that unit tests cannot find, such as missing configuration, a broken start-up, or a failed database connection.
- **Sanity testing:** Sanity testing is done manually after a small fix. Instead of running every test again, the team re-checks only the affected behaviour and its neighbours. For example, after a change to the double-booking rule, the team re-runs the related test cases (TC-03 and TC-04) and then tries one booking in the interface.

Evidence:

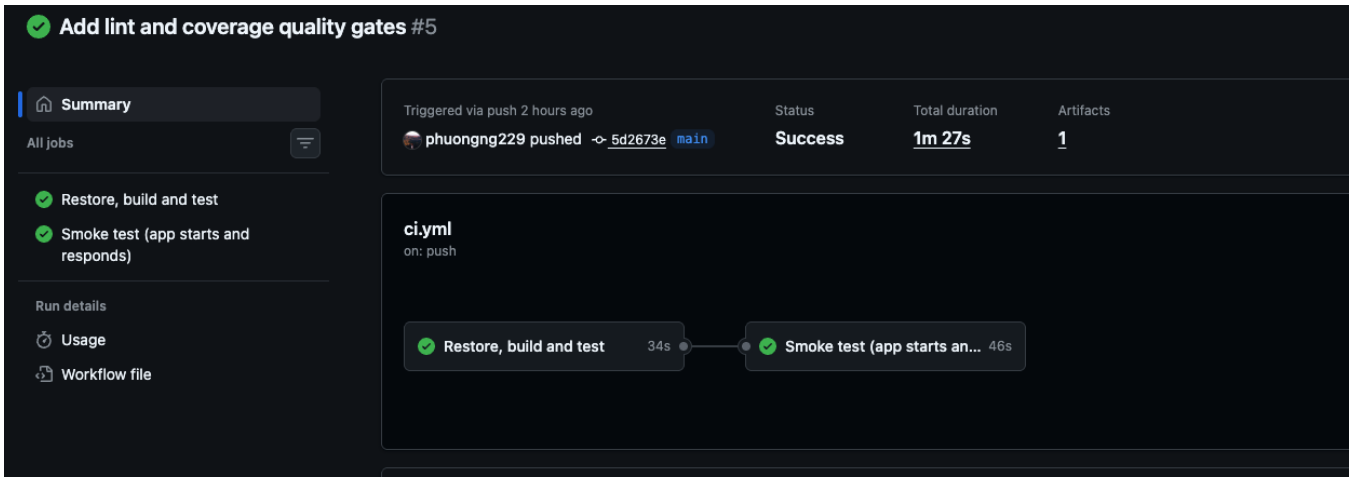


Figure 16: Both tests passed

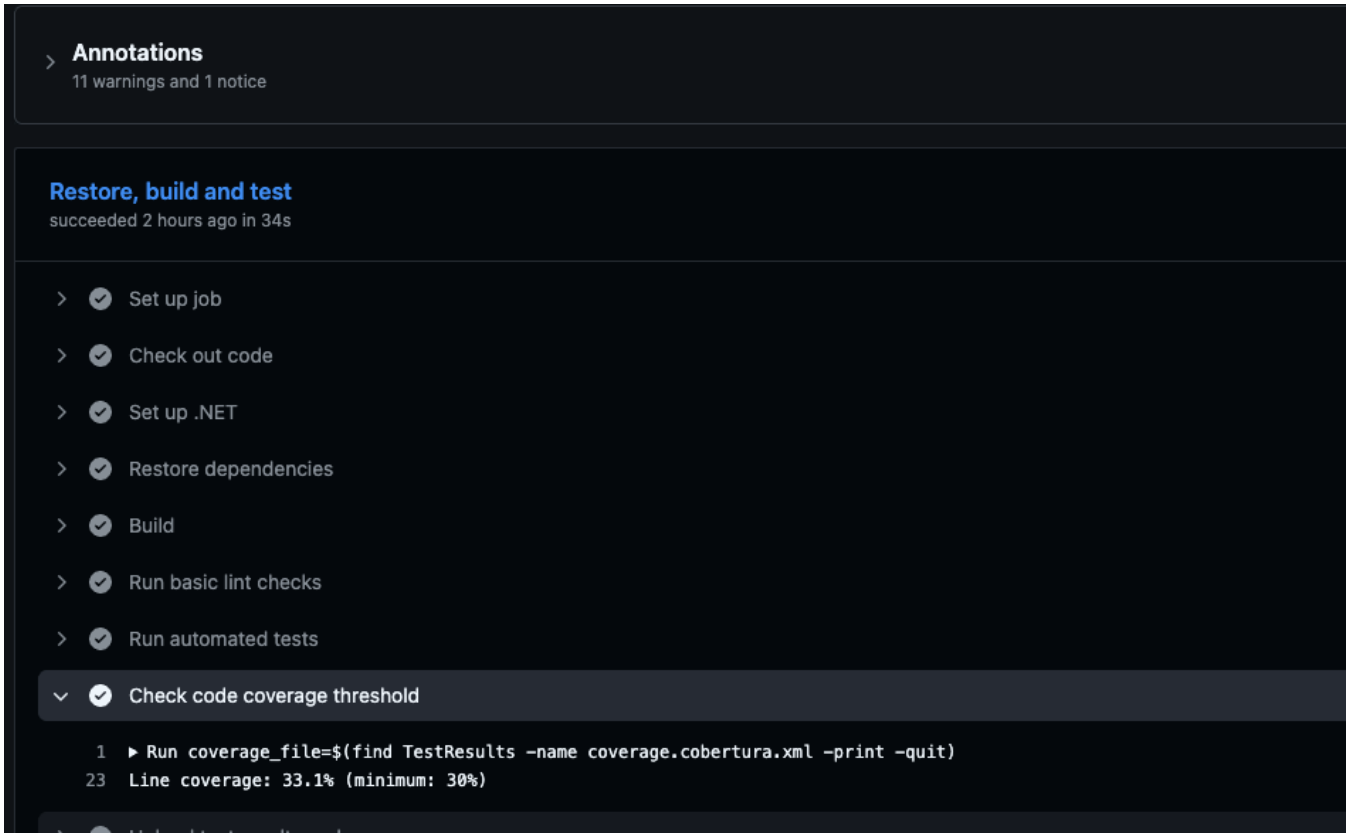


Figure 17: Checking code coverage threshold screenshot



Figure 18: Running automated test

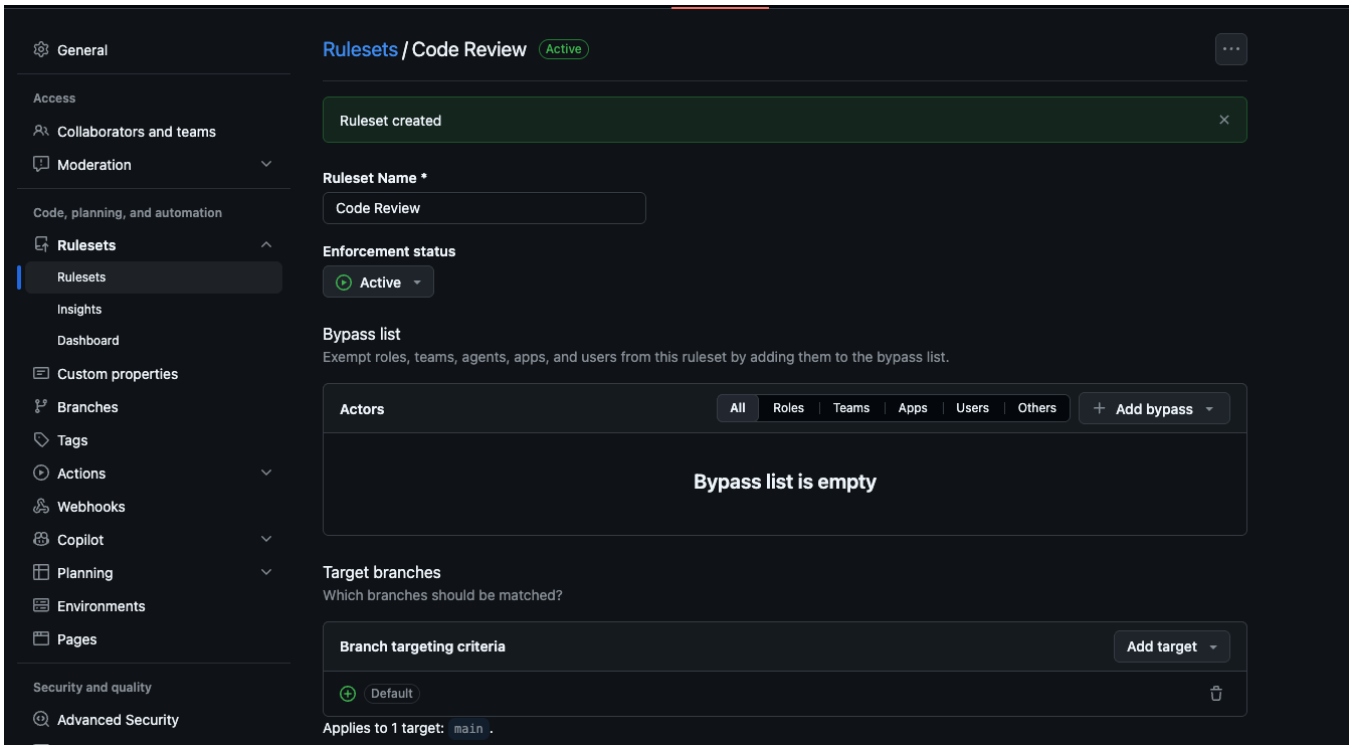


Figure 19: Code review ruleset setup 1

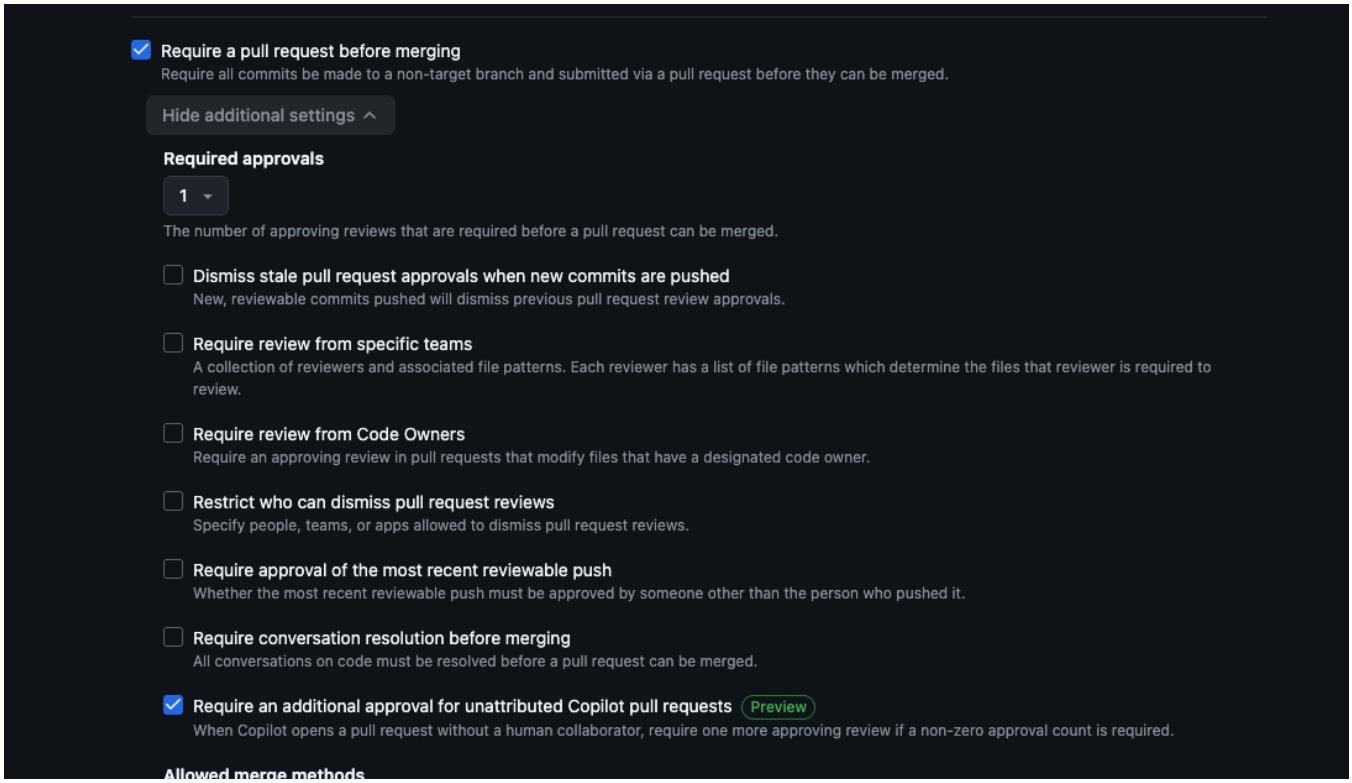


Figure 20: Code review ruleset setup 2

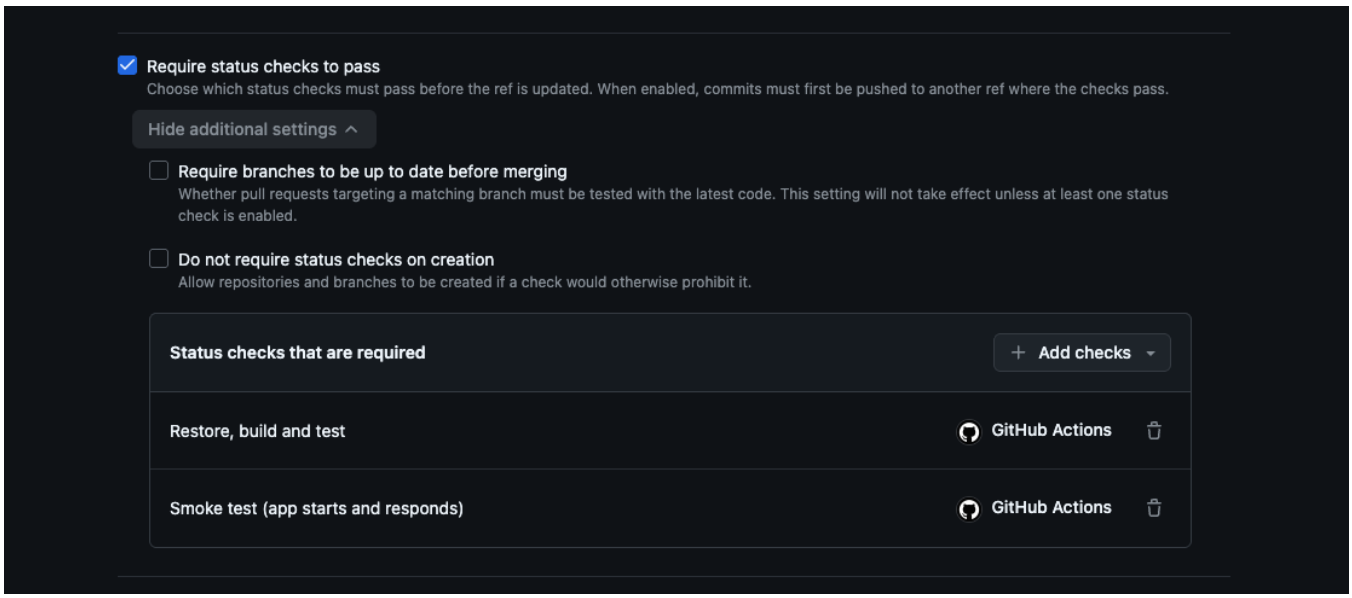


Figure 21: Code review ruleset setup 3

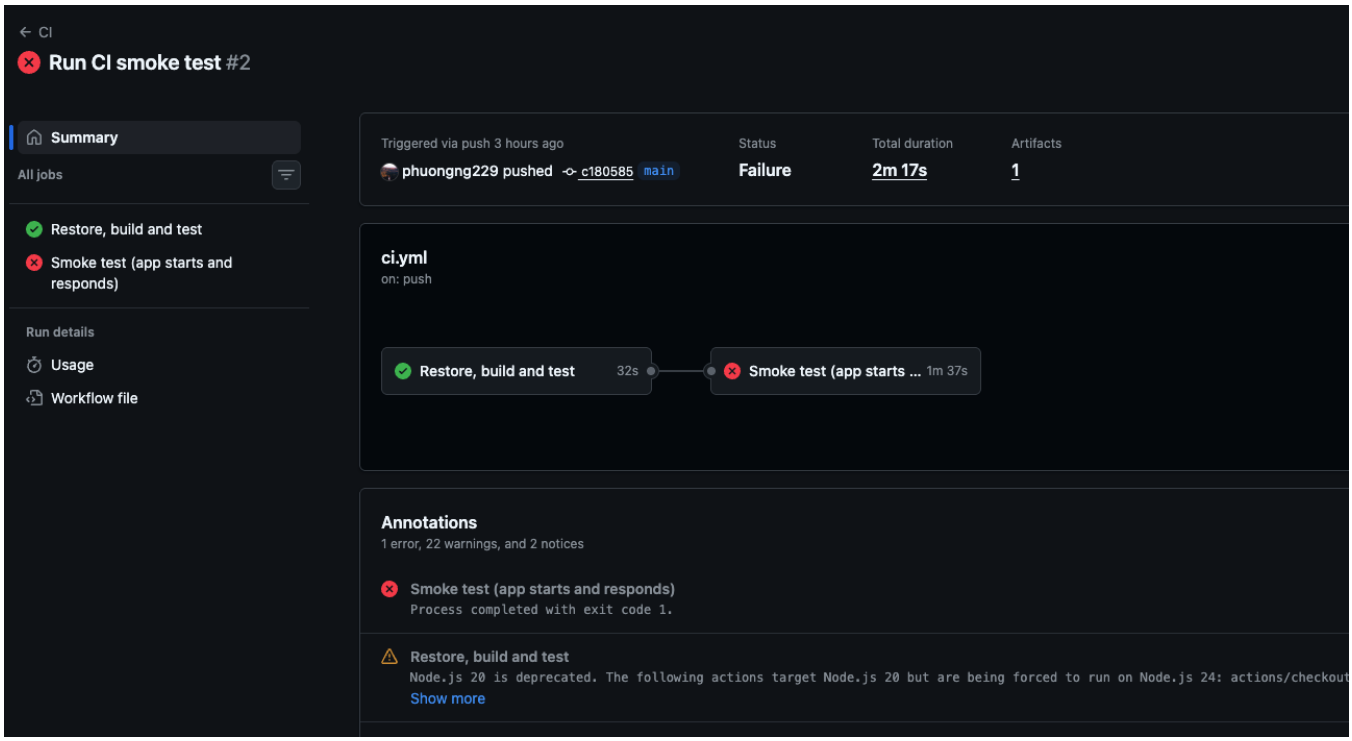


Figure 22: A failed run

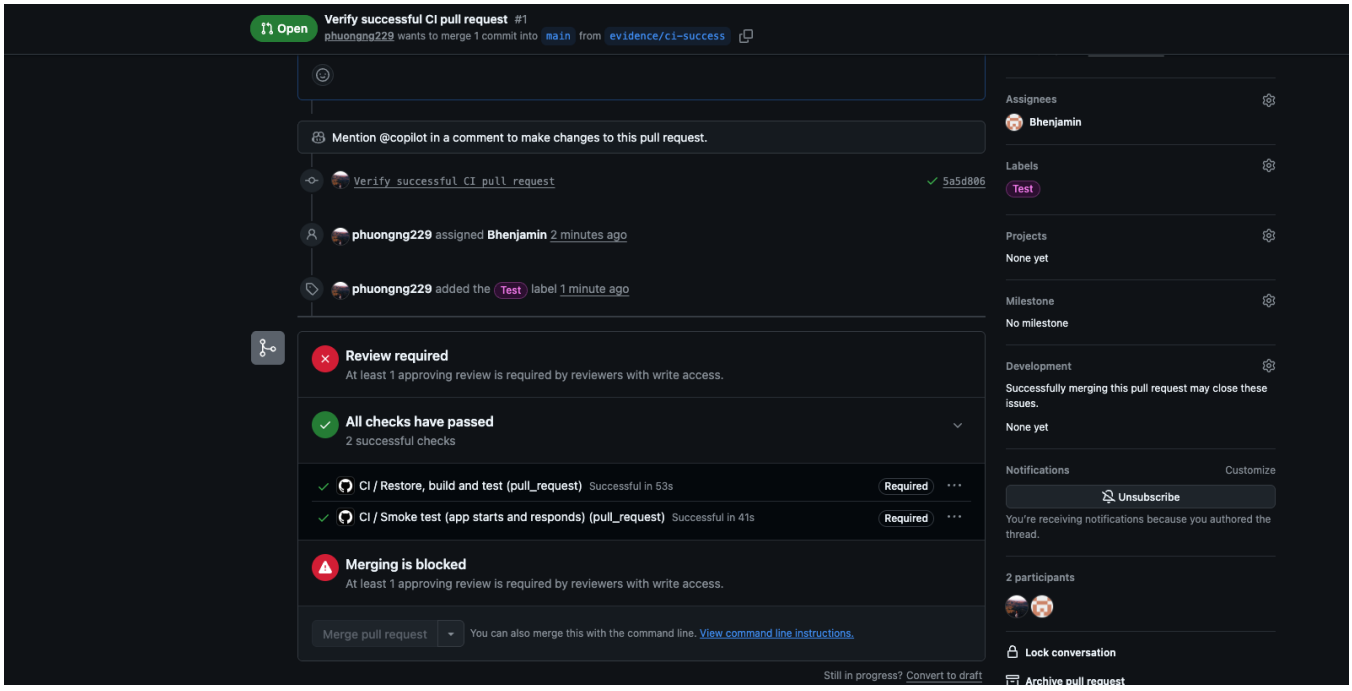


Figure 23: Create pull request test

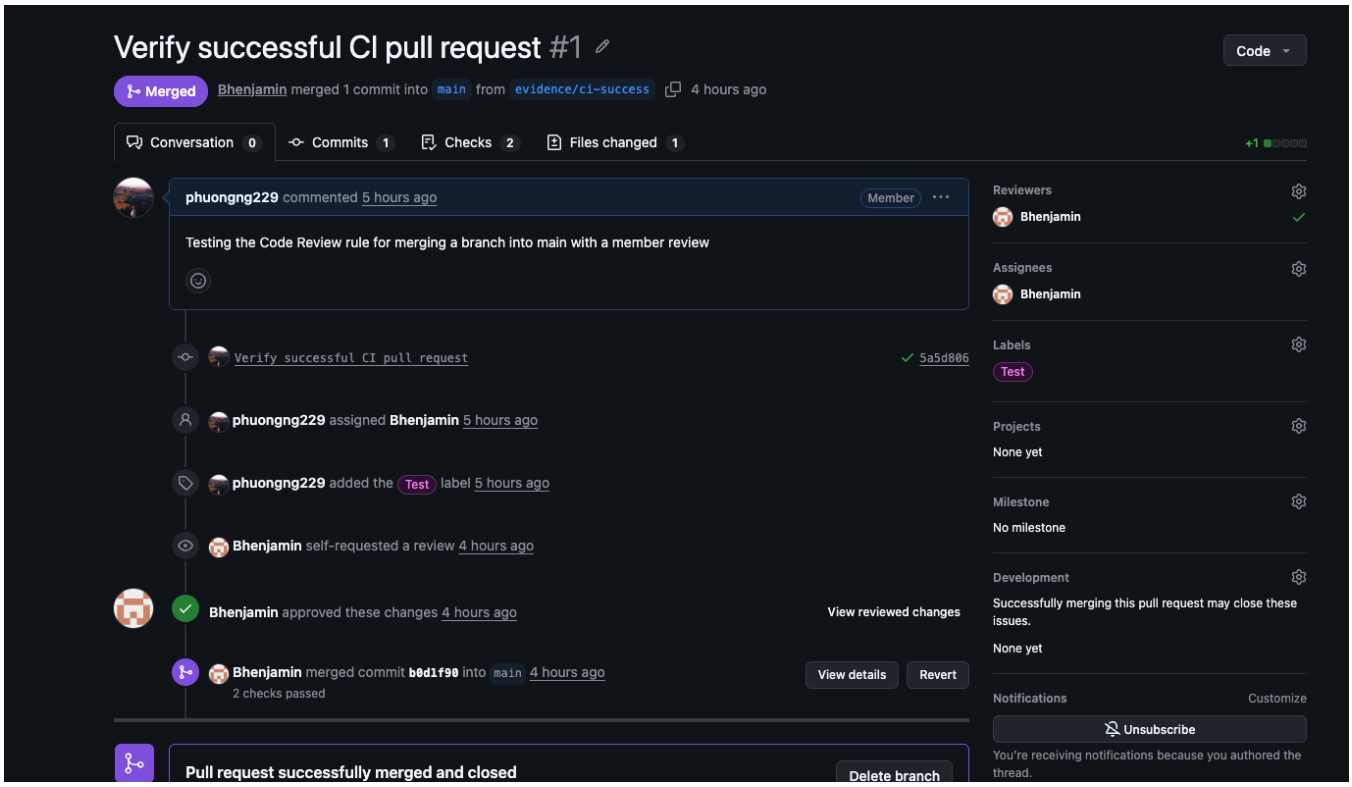


Figure 24: Approved pull request by 1 reviewer

6.5. Limitations and Future Improvements

- The pipeline does not deploy the application because the prototype is only used for demonstrations.
- The coverage threshold is low, and the lint check is basic. Both could be made stricter.
- The smoke test uses a real external database (Supabase), so it can fail if that service is down, even when the code is correct. A separate test database would be safer.
- Repository secrets are not available to pull requests from forks, which is acceptable for a three-person team.
- UI, email, recurring-booking and language-switching tests remain manual.
- The smoke test checks only the home page, not full booking flows.

7. Defect Management and Root Cause Analysis

7.1. Defect Management Approach

Defect management was used throughout the development and testing of the system to identify, record, investigate, fix, and verify software defects. Defects were identified through automated MSTest execution, manual system testing, regression testing, and code review. After a correction was implemented, the relevant test cases were rerun to verify the fix and check that existing functionality remained unaffected.

Each defect was assessed according to its severity and priority. Severity describes the impact of a defect on the system, while priority indicates how urgently it should be fixed. This distinction helped the team focus on issues that could affect booking reliability, access control, data integrity, or user understanding.

Category	Level	Definition
Severity	Critical	Causes system-wide failure, major data loss, or a serious security breach.
Severity	High	Breaks an important function, such as booking validation or access control.
Severity	Medium	Affects a feature or user experience, but a workaround is available.
Severity	Low	Causes a minor display, wording, or usability issue.
Priority	P1 – Urgent	Must be fixed immediately because of significant operational or security risks.
Priority	P2 – High	Should be fixed before the final release or demonstration.
Priority	P3 – Normal	Should be addressed when scheduled development work allows.
Priority	P4 – Low	Can be deferred if it has limited impact.

Table 14: Defect severity and priority definitions

Defect status describes the current stage of a defect:

- Open: The defect has been identified and recorded but has not been fixed.
- In Progress: A developer is investigating or correcting the defect.
- Fixed: A code or configuration change has been implemented.
- Retest: The correction is being verified through testing.
- Closed: Testing confirms the defect has been resolved.
- Deferred: The defect or associated feature has been postponed to a future iteration.

7.2. Final Defect Register

The following register summarises three significant defect examples relevant to the project's development and testing.

ID	Defect and Impact	Severity	Priority	Corrective Action	Status
DEF-01	Booking validation and database integration could allow incorrect	High	P1	Reviewed the booking validation	Closed, verified through

	booking behaviour if availability checks were not applied consistently after the database migration. This could affect booking reliability and create conflicting reservations.			logic, added automated tests for overlapping bookings, and reran regression tests after the database changes.	regression testing
DEF-02	The English word “Free” on the room calendar could be interpreted by Vietnamese users as meaning that a room is free of charge rather than being available. This could confuse users when selecting a room.	Low	P2	Reviewed the translation with manual testing and changed the wording to communicate room availability more clearly.	Closed, verified through UI and language testing
DEF-03	Changes to application configuration or database connectivity could cause the application to fail during startup even when individual unit tests passed. This risk became more important when the application was integrated with the actual database.	High	P2	Added a GitHub Actions smoke test to start the application, check its HTTP response, and report application logs if startup failed.	Closed, verified through a successful GitHub Actions CI run

Table 15: Final Defect Register

DEF-01 addresses booking reliability during the migration from in-memory repositories to persistent database storage. DEF-02 represents a usability and localisation issue identified through manual testing. DEF-03 concerns integration reliability and the gap between isolated tests and the complete running application.

The statuses above describe the intended resolution state and should only be retained if supported by test results, code changes, or CI evidence. If the team did not experience DEF-01 or DEF-03 as actual defects, replace them with verified issues from the GitHub commit history, test logs, or pull requests.

7.3. Root Cause Analysis

i. DEF-01: Booking Validation and Database Integration

Problem: Booking operations must reject overlapping reservations and preserve the booking rules after migrating from in-memory repositories to a database. Inconsistent validation between application services and database operations could allow conflicting bookings or incorrect booking results.

Root cause: The architectural change introduced an additional integration boundary between the booking logic and persistent data storage. If the availability check and booking creation are not coordinated correctly, a booking could be created using outdated availability information. The migration therefore required regression testing and verification of the booking workflow against the new persistence implementation.

Corrective action: The team used MSTest with an isolated in-memory SQLite database to test successful bookings, overlapping bookings, cancellation, modification, and invalid booking requests. The original booking tests were rerun after the database migration, and TC-20 was used to verify double-booking prevention.

Prevention measures:

- Keep booking validation in a clearly defined service rather than duplicating rules across multiple components.
- Use database transactions and suitable database constraints where appropriate to protect booking integrity.
- Add automated regression tests whenever booking logic or persistence code changes.
- Test concurrent booking requests and the actual database provider before production deployment.

Quality improvement: This analysis reinforced the importance of regression testing after architectural changes. It also demonstrated that testing service logic alone is not sufficient to establish that booking integrity is preserved across the complete database integration.

ii. DEF-02: Ambiguous Vietnamese Translation

Problem: The room calendar originally used “Free” to indicate that a room was available. In Vietnamese, the translated wording could be interpreted as indicating that the room is free of charge rather than available for booking.

Root cause: The wording was selected based on its English meaning without sufficiently considering how the translated term would be interpreted in the booking context. Literal translation did not communicate the intended state clearly enough to users.

Corrective action: During manual language testing, the team reviewed the calendar wording and identified the ambiguity. The wording was changed to make room availability clearer. The language switch was then checked to confirm that the interface displayed the intended text in both English and Vietnamese without requiring an application restart.

Prevention measures:

- Review translated interface text in its actual user-interface context.
- Prefer clear, consistent terminology for room states, such as “Available” and “Booked.”
- Ask a proficient speaker to review translations of important booking actions and statuses.
- Include localisation and language-switching checks in the regression test checklist.

Quality improvement: This defect demonstrated that correct application logic does not guarantee good usability. Manual testing identified a problem that a functional test checking only whether the language changed would not necessarily detect. The correction improved clarity and strengthened the team's approach to localisation testing.

iii. DEF-03: Application Startup and Database Configuration

Problem: Automated unit tests can pass even when the complete application cannot start because of missing configuration, an invalid database connection, or an integration problem. Such a failure could prevent the team from demonstrating the application despite successful service level tests.

Root cause: Unit tests execute selected application components in a controlled environment. They do not necessarily validate application startup, deployment configuration, or connectivity to the actual database. This created a gap between component level verification and end-to-end application readiness.

Corrective action: The team implemented a GitHub Actions workflow containing a smoke test that runs after the build, and test succeeds. The smoke test starts the web application, waits for an HTTP response, and outputs application logs if startup fails. The database connection string is supplied through a repository secret rather than being hard coded in the source code.

Prevention measures:

- Run the smoke test after successful builds and automated tests.
- Keep database credentials in repository secrets and avoid exposing them in logs.
- Use a dedicated test database where possible to reduce reliance on external services.
- Test important application startup and configuration paths before demonstrations and releases.
- Record CI failures and rerun checks after corrective changes.

Quality improvement: The smoke test extended quality assurance beyond unit testing by checking whether the application could start and respond in its integrated environment. It also provided repeatable evidence of application readiness and made configuration-related failures easier to investigate.

7.4. Defect Lifecycle

The team follows a defect lifecycle that connected testing, development, code review, and regression testing.

1. Identification: A potential defect is discovered through an automated test, manual testing, code review, or integration check.
2. Recording and classification: The defect is documented with its symptoms, affected feature, severity, and priority.
3. Investigation: The responsible developer reproduces the issue where possible and investigated the underlying cause.
4. Correction: A code, configuration, or interface change is implemented on a feature branch.
5. Review and verification: The change is reviewed through a pull request, and relevant automated or manual tests were rerun.
6. Regression testing: Related tests are executed to check that the correction had not broken existing functionality.
7. Closure or deferral: The defect is closed when verification supported the fix. Unresolved issues were kept open or formally deferred with their risks documented.

GitHub pull requests and automated CI checks supports this process. Build failures, failed tests, and smoke-test failures provided feedback before code is merged into the main branch. However, a green CI result alone is not considered proof that every usability, accessibility, or external email-delivery issue has been resolved; these areas still require appropriate manual verification.

7.5. Corrective Actions and Defect Prevention

The project used both corrective and preventive approaches. Corrective actions addressed identified problems, while preventive measures aimed to reduce the likelihood of similar defects occurring again.

Area	Corrective Action	Preventive Measure
Booking reliability	Fix validation or persistence issues and rerun booking tests	Maintain automated tests for overlapping bookings, modification, cancellation, and invalid inputs
Localisation and usability	Replace ambiguous room-status wording	Review translations in context and include language checks in regression testing
Integration reliability	Investigate startup or database connection failures	Run CI smoke tests and keep configuration and credentials separate from source code
Code quality	Correct issues identified during review or automated checks	Use feature branches, pull-request reviews, build validation, and automated tests
Test reliability	Repeat affected tests after a correction	Use isolated test data and databases to make results repeatable and reduce test interference

Table 16: Corrective actions and defect prevention

These measures are proportionate to the project's scope and provide a foundation for further quality improvements. Future work should include testing against the actual

database provider, more comprehensive accessibility checks, and a more complete end-to-end automated test suite.

7.6. Evaluation of Defect Management

Defect management improved the project by connecting identified problems with corrective actions and verification evidence. Booking-related testing helped protect core business rules during the database migration. Manual localisation testing identified an ambiguity that automated functional tests could have missed. The CI smoke test added an integration level check to detect startup and configuration failures before a demonstration or merge.

The process also highlighted limitations. The isolated SQLite test database may behave differently from the application's actual database, and the smoke test does not verify complete booking workflows. In addition, the team should retain reproducible defect records and supporting evidence so that the final register reflects actual defects rather than possible risks.

Overall, combining automated regression testing, manual user-interface testing, root cause analysis, code review, and CI checks strengthened the reliability and maintainability of the prototype. The main lesson was that quality assurance must address not only whether individual functions work, but also whether integrated features remain reliable, understandable, and verifiable after changes.

8. Quality Testing

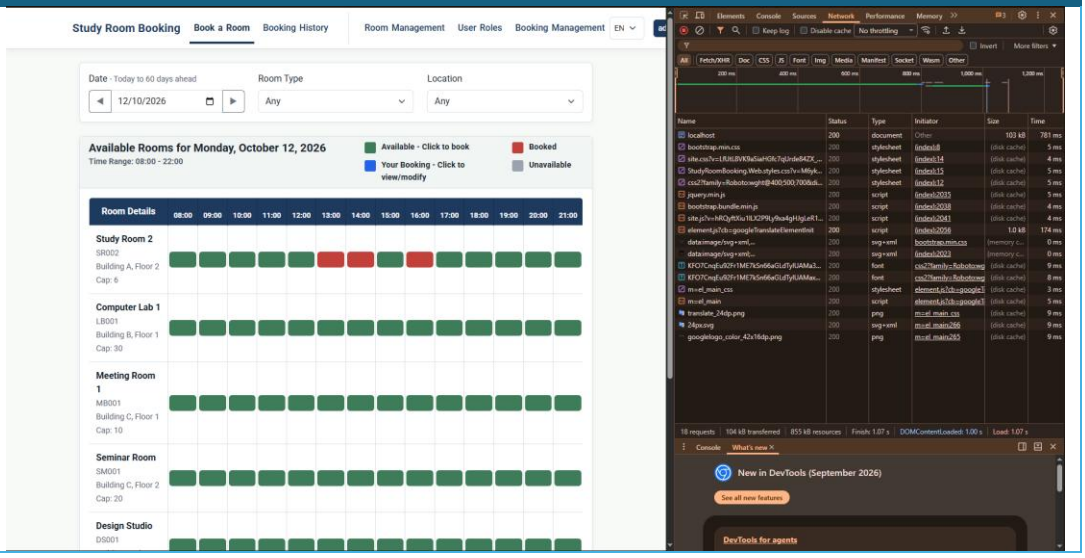
Quality testing was conducted to evaluate the system against relevant quality requirements, including performance, usability, accessibility, compatibility, reliability, security, and maintainability.

8.1. Performance Quality Testing

NFR1 - Room searches will return results within 2 seconds for at least 50 reservations

Goal	To determine whether the room available room reservations for a particular day return within 2 seconds.
Question	Does search return within 2 seconds with at least 50 reservations?
Metric	The response time of the local host request using dev tools in a chromium-based browser
Interpretation	All test results were below the 2000ms threshold showing 98 reservations Average time of 721ms Max time of 781ms Min time of 622ms Slowest run was 781ms against a 2000ms limit, so there's a margin of over 1.2 seconds, which suggests that the system comfortably meets the performance requirement, as well as this the automated test case (TC 17) passes.
Decision	Passes as all tested times are under the threshold

Table 17: Performance quality testing results for NFR1

Search Results load time	Screenshots of times at 98 reservations
781ms	

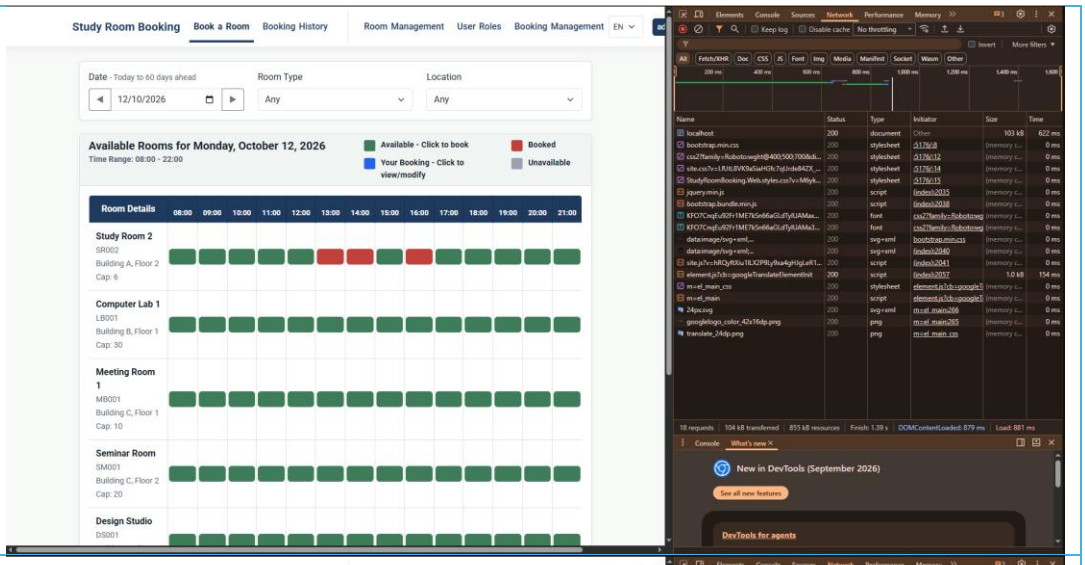
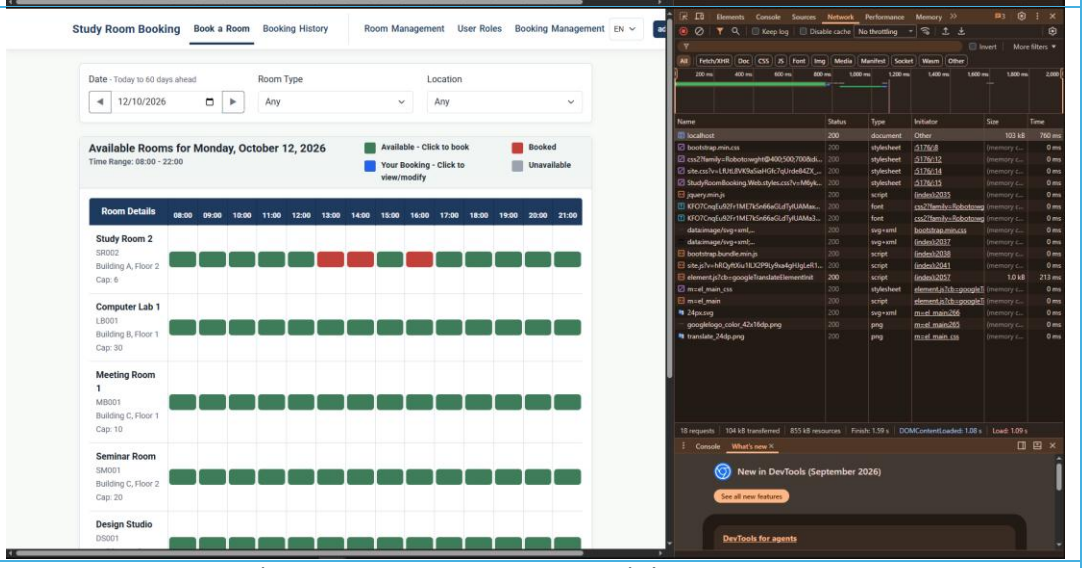
622ms	
760ms	
Average	(781ms + 622ms + 760ms) / 3 = **721ms**

Table 18: Room search response time measurements with 98 reservations

Limitations:
These tests were done on a locally hosted server, which would not accurately reflect the performance of the system in a real-world deployment environment. In a deployed environment, latency may vary depending on the user's location relative to the server as well as other network conditions such as server load. Because of this it was difficult to test performance realistically, but the results still provide an indication of how efficiently our code is retrieving the data.

Recommended improvements:
Once this is deployed, the search performance could be measured under realistic operating conditions. Further testing could also be done assess how search performance is affected by an increasing number of concurrent users. This would help determine how well the system performs as while lots of students are using the booking system.

Automated tests:

TC 17: This unit test tests the retrieval speed of the data using a timer and determines whether that is less than 2000ms. As this test case passes it proves the code can retrieve the data from the in-memory test database within 2000ms.

```
// TC-17: Room search returns results within 2 seconds for 50+ reservations (NFR1)
// Verifies that the search performance meets the requirement of < 2 seconds
// when searching among 50+ existing bookings.
[TestMethod]
0 references
public async Task TC17_RoomSearch_Returns50Results_WithinTwoSeconds()
{
    // Arrange
    var rooms = new List<Room>();
    var allBookings = new List<Booking>();
    for (int i = 1; i <= 10; i++)
    {
        await AddUserAsync(i);
    }

    // Create 60 rooms with multiple bookings each
    for (int i = 1; i <= 60; i++)
    {
        rooms.Add(new Room
        {
            Id = i,
            Code = $"TEST-{i}",
            Name = $"Room {i}",
            Capacity = 4,
            Type = i % 2 == 0 ? RoomType.Study : RoomType.Meeting,
            IsAvailable = true
```

Figure 25: TC-17 test code

```
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance> cd .\StudyRoomBooking\
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance\StudyRoomBooking> dotnet test
Restore complete (0.9s)
  StudyRoomBooking.Domain net8.0 succeeded (0.2s) → src\StudyRoomBooking.Domain\bin\Debug\net8.0\StudyRoomBooking.Domain.dll
  StudyRoomBooking.Application net8.0 succeeded (0.2s) → src\StudyRoomBooking.Application\bin\Debug\net8.0\StudyRoomBooking.Application.dll
  StudyRoomBooking.Infrastructure net8.0 succeeded (0.3s) → src\StudyRoomBooking.Infrastructure\bin\Debug\net8.0\StudyRoomBooking.Infrastructure.dll
  StudyRoomBooking.Tests net8.0 succeeded (0.3s) → tests\StudyRoomBooking.Tests\bin\Debug\net8.0\StudyRoomBooking.Tests.dll
  StudyRoomBooking.Tests test net8.0 succeeded (1.9s)

Test summary: total: 12, failed: 0, succeeded: 12, skipped: 0, duration: 1.8s
Build succeeded in 3.7s
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance\StudyRoomBooking>
```

Figure 26: Test results

Limitation:

However, this does not account for latency and is limited in that it uses an in-memory SQLite database to protect our data.

Recommended improvements:

Continuing to do manual testing through the network test through a browser and in future development look into a way to automate doing the network tests. So that data retrieval from the actual server can be tested under more realistic conditions without interfering with existing data.

8.2. Security Quality Testing

NFR3 - Only authorised users may access administrative functions

Goal	To determine whether non-admin roles can access/use admin features
Question	Can a non-admin user access admin pages through URLs even though they don't have permission?
Metric	Whether the non-admin user can access and perform admin functions

Interpretation	9/9 of the URL tests passed. The automated test case (TC 15) also passes.
Decision	All test passed against NFR3, but there are many ways of breaching security that are not accounted for, see limitations for more details.

Table 19: Security quality testing results for NFR3

The testing for quality test this was trying to access the Admin Room management page for while logged in student and staff. this was done by pasting this link in the URL for the different admin pages and seeing if user could access this from a student or staff account or if without logging in. Before these tests the admin links were tested first while logged in as an admin to ensure that they worked, so if a student or staff member couldn't access the pages, it was because they didn't have permission and not because the links were wrong.

Tested URL	Were they able to access admin functions
http://localhost:5176/Admin/RoomManagement	**Student:** unable to access **Staff:** unable to access **No Login:** unable to access
http://localhost:5176/Admin/UserRoles	**Student:** unable to access **Staff:** unable to access **No Login:** unable to access
http://localhost:5176/Admin/BookingManagement	**Student:** unable to access **Staff:** unable to access **No Login:** unable to access

Table 20: Administrative URL access tests by user role

Every test directed students/staffs back to the home/booking page (when logged in) or login page (when not logged in) as they should not have access. Proving that the access role checking was working correctly, and unqualified roles cannot access admin functionality by changing the URL.

Automated tests:
TC-15: This is can also been seen in this TC-15, which automatically tests whether non-admin roles can call admins functions at the code level. As this test case passes this confirms that the logic for admins role access is functionally correct.

```
// TC-15: Non-administrator cannot cancel another user's booking (FR11, NFR3)
// Verifies that only the booking owner or admin can cancel a booking.
[TestMethod]
[ExpectedException(typeof(UnauthorizedAccessException))]
0 references
public async Task TC15_NonAdmin_CannotCancelOtherUserBooking()
{
    // Arrange
    await AddUserAsync();
    await AddRoomAsync();
    var booking = new Booking
    {
        Id = 1,
        UserId = 10, // Owner is user 10
        RoomId = 1,
        BookingDate = DateTime.Today.AddDays(1),
        Status = BookingStatus.Confirmed
    };
    await AddBookingAsync(booking);

    // Act: User 20 (non-admin, different from owner) tries to cancel user 10's booking
    // This should throw UnauthorizedAccessException
    await _bookingService.CancelBookingAsync(1, 20, false);

    // Assert: Exception thrown (handled by [ExpectedException])
}
```

Figure 27: TC-15 test code

```
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance> cd .\StudyRoomBooking\
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance\StudyRoomBooking> dotnet test
  Restore complete (0.9s)
    StudyRoomBooking.Domain net8.0 succeeded (0.2s) → src\StudyRoomBooking.Domain\bin\Debug\net8.0\StudyRoomBooking.Domain.dll
    StudyRoomBooking.Application net8.0 succeeded (0.2s) → src\StudyRoomBooking.Application\bin\Debug\net8.0\StudyRoomBooking.Application.dll
    StudyRoomBooking.Infrastructure net8.0 succeeded (0.3s) → src\StudyRoomBooking.Infrastructure\bin\Debug\net8.0\StudyRoomBooking.Infrastructure.dll
    StudyRoomBooking.Tests net8.0 succeeded (0.3s) → tests\StudyRoomBooking.Tests\bin\Debug\net8.0\StudyRoomBooking.Tests.dll
    StudyRoomBooking.Tests test net8.0 succeeded (1.9s)

  Test summary: total: 12, failed: 0, succeeded: 12, skipped: 0, duration: 1.8s
  Build succeeded in 3.7s
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance\StudyRoomBooking>
```

Figure 28: Test results

Limitations:
The limitations of these testing methods are that they do not account for the wide range of ways people could attempt to breach security and access admin functionality. Examples could be some of the following:

- Direct requests to admin actions
- Tampering with login or session data
- Input validation (e.g. SQL injection)
- Weak passwords

Our current test suite doesn't account for some of the critical ways people can gain unauthorised access.

Recommended improvements:
Before this is deployed for the university students to use, counter measures and test cases should be implemented to address the above security threats.

9. Quality Metrics and Final Release Decision

9.1. Test Pass Rate

See full evidence in of testing in section **4.2** and section **8**

Automated Test Pass Rate: 12/12 = 100%

```
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance\StudyRoomBooking> dotnet test
Restore complete (0.9s)
  StudyRoomBooking.Domain net8.0 succeeded (1.7s) → src\StudyRoomBooking.Domain\bin\Debug\net8.0\StudyRoomBooking.Domain.dll
  StudyRoomBooking.Infrastructure net8.0 succeeded (0.7s) → src\StudyRoomBooking.Infrastructure\bin\Debug\net8.0\StudyRoomBooking.Infrastructure.dll
  StudyRoomBooking.Application net8.0 succeeded (0.7s) → src\StudyRoomBooking.Application\bin\Debug\net8.0\StudyRoomBooking.Application.dll
  StudyRoomBooking.Tests net8.0 succeeded (0.6s) → tests\StudyRoomBooking.Tests\bin\Debug\net8.0\StudyRoomBooking.Tests.dll
  StudyRoomBooking.Tests test net8.0 succeeded (2.1s)

Test summary: total: 12, failed: 0, succeeded: 12, skipped: 0, duration: 2.1s
Build succeeded in 6.4s
(base) PS C:\Users\Rename Me Pls\Documents\Software-Quality-Assurance\StudyRoomBooking>
```

Figure 29: Automated test results

Manual Test Pass Rate: 8/8 = 100%

Test ID	Test Type	Requirement	Test Scenario	Execution Method	Result
TC-09	Functional	FR1	Search using all selected room filters	Manual	**Pass**
TC-10	Negative / Boundary	FR1	Search with criteria that match no rooms	Manual	**Pass**
TC-11	Functional	FR5	Send confirmation notification with correct booking details	Manual	**Pass**
TC-13	System / Functional	FR9	Create recurring weekly bookings and handle a conflict	Manual	**Pass**
TC-16	System / Security	FR11, NFR3	Administrator overrides and cancels another booking	Manual	**Pass**
TC-18	Usability / Acceptance	NFR4	First-time user completes a booking within 4 minutes	Manual	**Pass**
TC-19	Accessibility / Language	NFR6	Switch between Vietnamese and English without restarting the application	Manual	**Pass**
TC-20	Integration / Regression	NFR2, FR3	Re-verify double-booking prevention after database migration	Manual	**Pass**

Table 21:Manual test results

Test Pass Rate: 20/20 = **100%**

9.2. Defect Severity Distribution

See section 7.2 final defect register for more information

Severity	Count	Defect IDs	Found by	Status
Critical	0	None	N/A	N/A
High	2	DEF-01, DEF-03	DEF-01: automated and regression tests. DEF-03: CI smoke test	Closed
Medium	0	None	N/A	N/A
Low	1	DEF-02	Manual language testing	Closed
Total	**3**			**3 closed**

Table 22: Defect severity distribution and resolution status

9.3. Requirement Coverage

ID	Requirement	Implemented	Test Cases	Result
FR1	Search rooms by filters	Yes	TC-09, TC-10	Pass
FR2	Book an available room	Yes	TC-01, TC-08	Pass
FR3	Validate availability before booking	Yes	TC-01, TC-02, TC-07, TC-20	Pass
FR4	Programme eligibility for specialised rooms	Yes	TC-03, TC-04	Pass
FR5	Booking confirmation email	Yes	TC-11	Pass
FR6	Modify future bookings	Yes	TC-05	Pass
FR7	Cancel future bookings	Yes	TC-06	Pass
FR8	View booking history	Yes	TC-12	Pass
FR9	Recurring bookings (staff)	Yes	TC-13	Pass
FR10	Admin manage rooms and access rules	Yes	TC-14	Pass
FR11	Admin override or cancel bookings	Yes	TC-15, TC-16	Pass
FR12	Room utilisation reports	No (deferred to next sprint)	None	Not tested

Table 23: Functional requirements

ID	Requirement	Test Cases/Metrics	Result
NFR1	Performance	TC-17 + manual testing (section 8)	Pass
NFR2	Reliability	TC-02, TC-20	Pass
NFR3	Security	TC-03, TC-04, TC-14, TC-15 + manual testing (section 8)	Pass
NFR4	Usability	TC-18	Pass
NFR5	Maintainability	See below Code Quality Indicators	Pass
NFR6	Accessibility	TC-19	Pass
NFR7	Availability	Observed this during testing and development, not a specific test case	Pass

Table 24: Non-functional requirements

Measure	Value
FRs implemented	11 / 12 = 91.67%
FRs with at least one passing test	11 / 12 = 91.67%
NFRs with at least one passing test	7 / 7 = 100%
All requirements with at least one passing test	18 / 19 = 94.74%

Table 25: Overall requirement coverage

9.4. Test Coverage

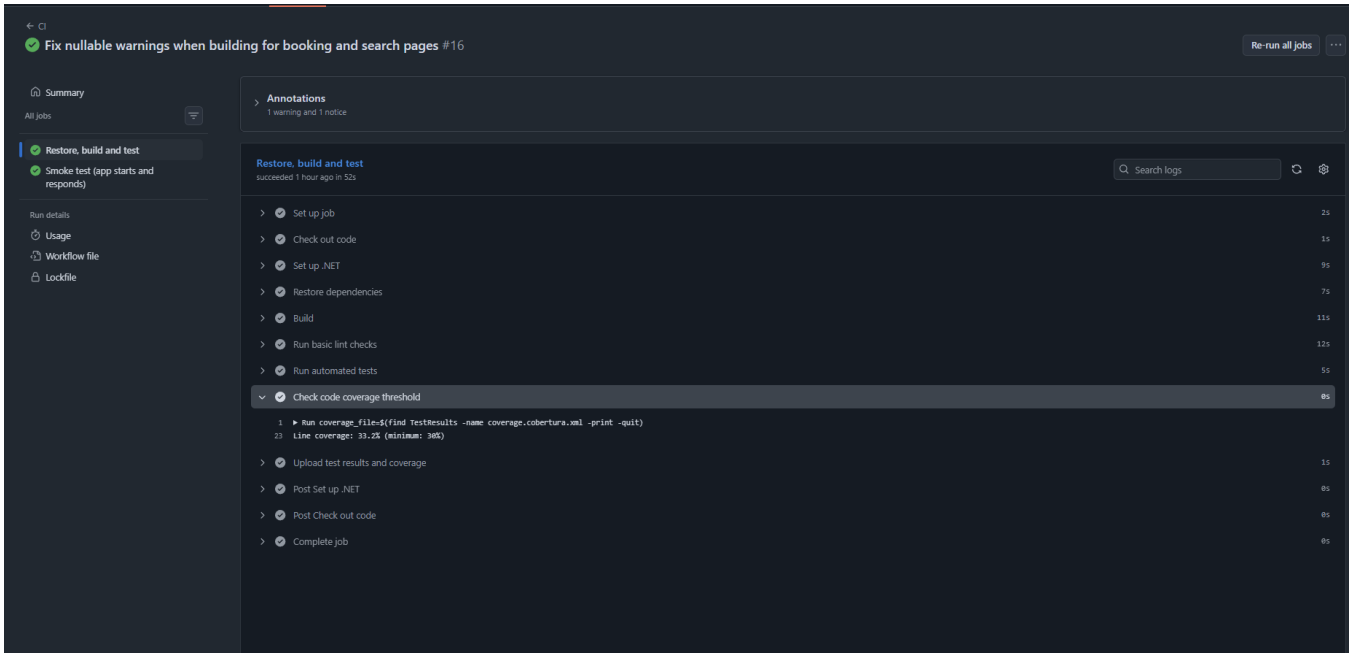


Figure 30: Test coverage result

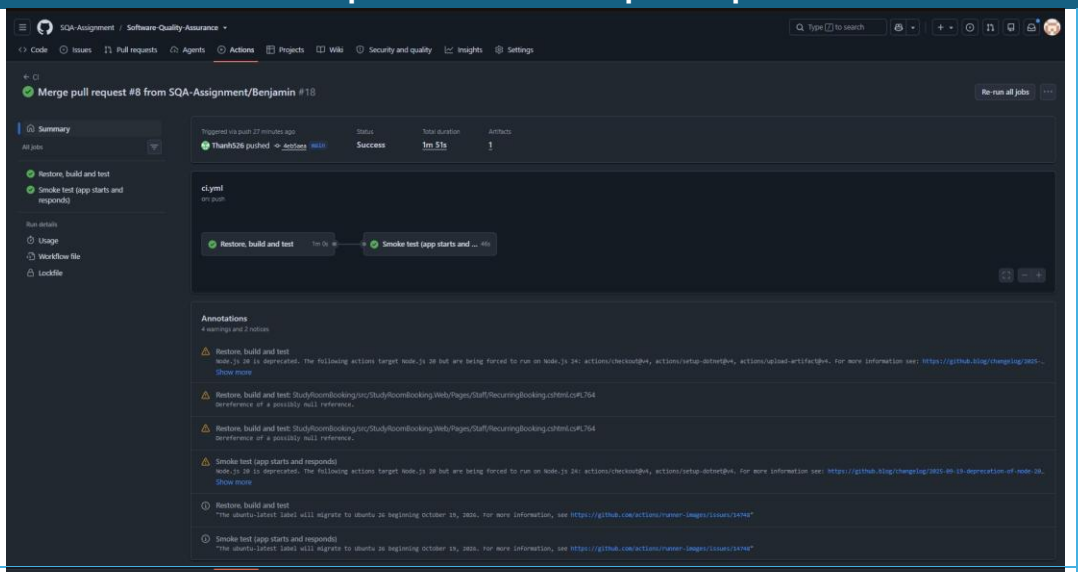
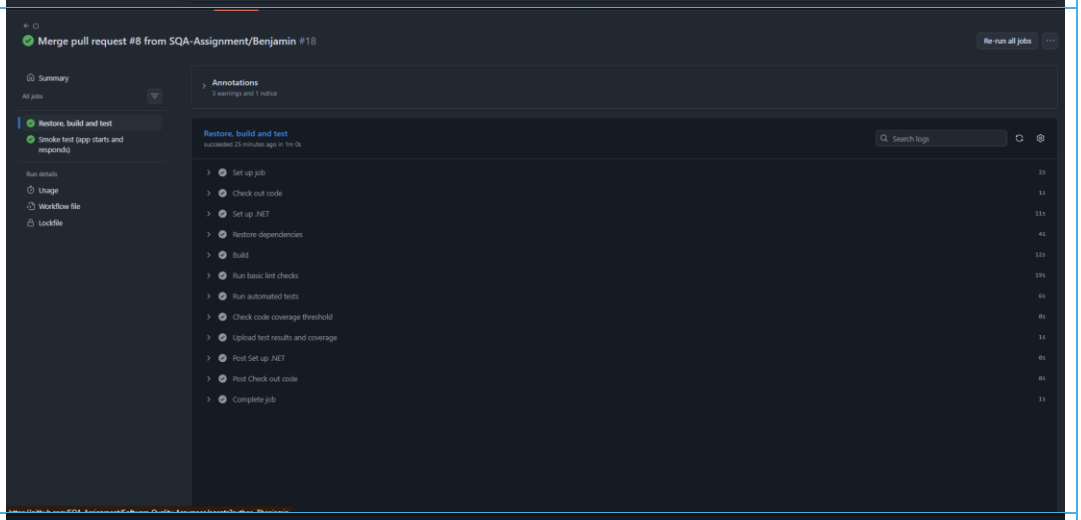
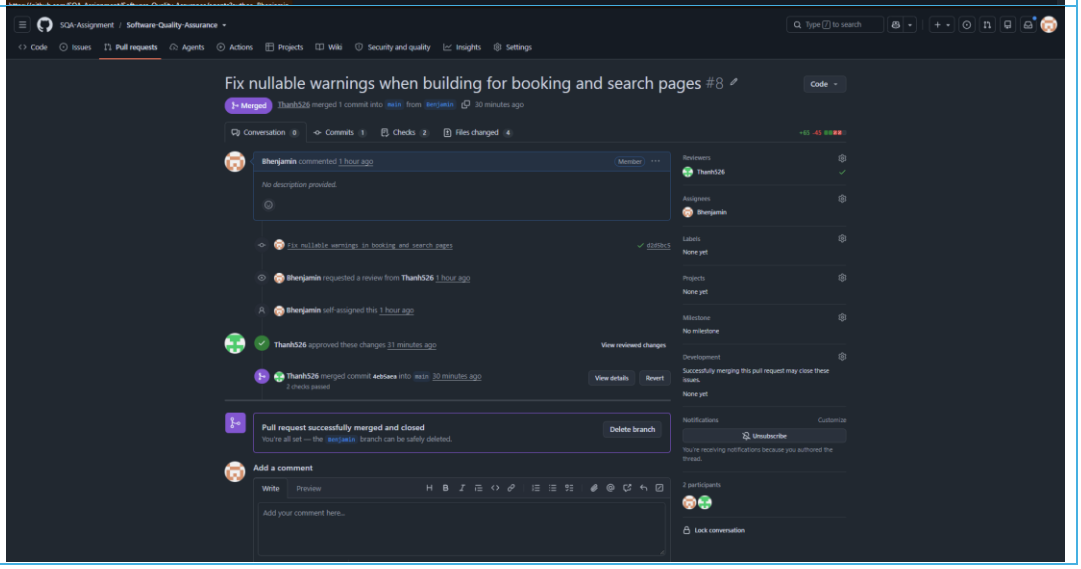
Test Coverage as of 17/10/2026:
Line coverage: 33.2%
CI threshold: 30%
Result: Passes threshold

9.5. Automation coverage

Tests	Amount
Automated Test	**12 automated test cases:** TC-01, TC-02, TC-03, TC-04, TC-05, TC-06, TC-07, TC-08, TC-12, TC-14, TC-15, TC-17
Manual Tests	**8 manual test cases:** TC-09, TC-10, TC-11, TC-13, TC-16, TC-18, TC-19, TC-20
Total	12 / 20 = 60% automated coverage

Table 26: Automated and manual test coverage

9.6. Code Quality Indicators

Indicator	Example of success on pull request
Build (Release): must compile with no errors	
Lint / analyser check: no analyser errors	
Code review: at least 1 approving review per pull request	

Lines of code	5959 lines of code

Table 27: Code quality indicators

9.7. Commit Activity

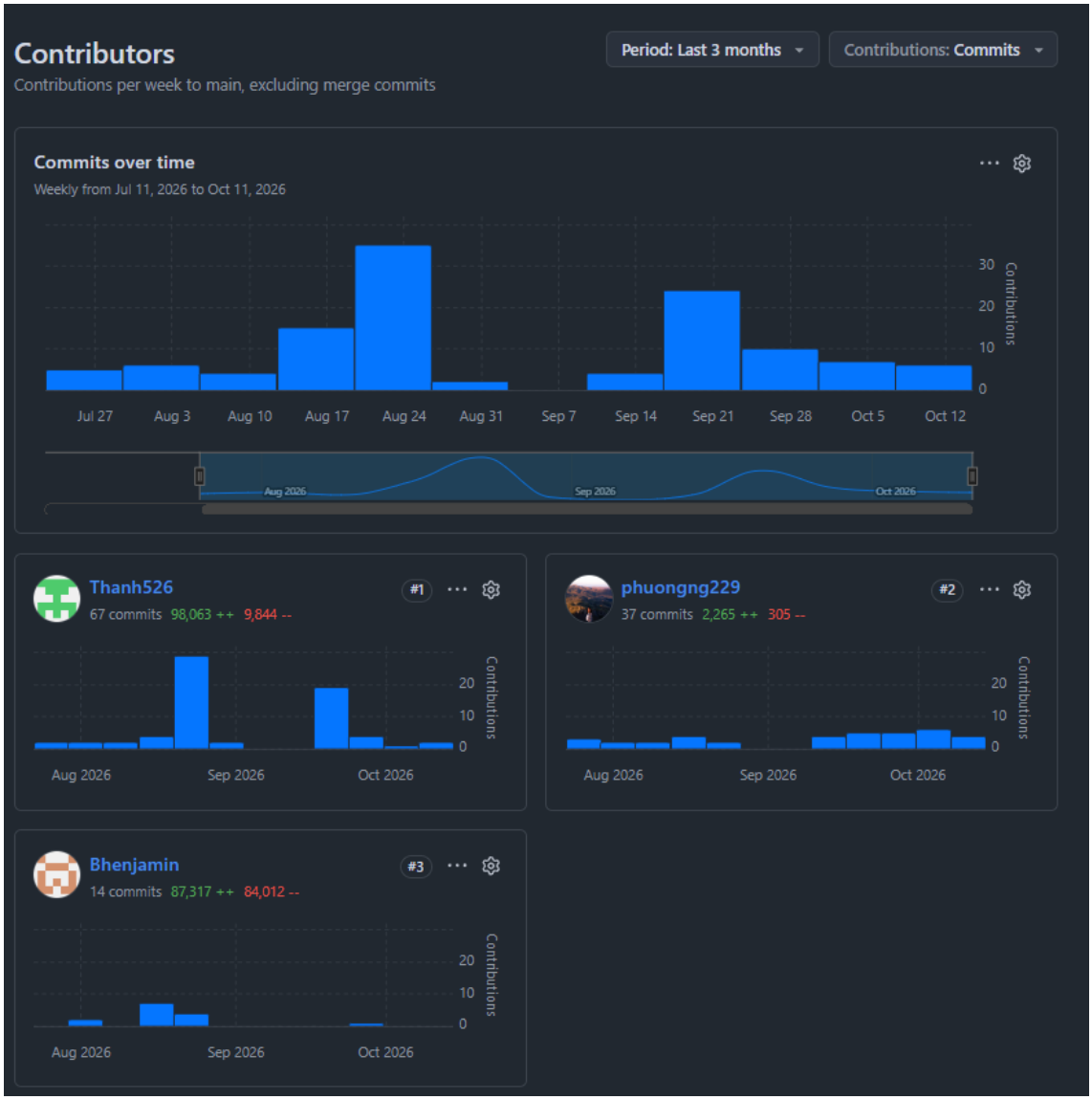


Figure 31: GitHub contributions and code change statistics

Member	Commits	Lines added	Lines deleted	Key Features delivered See more on table 1
Benjamin	15	87,382	84,057	Administrator Functions, GUI improvements
Phuong	37	2,265	305	Database, Email Confirmation
Thanh	68	98,124	9,891	Multiple Language Support, Normal booking, Recurring booking

Table 28: GitHub contributions and code change statistics

Results taken 17/10/2026

Note: Commit activity does not necessarily reflect the effort contributed, as each member's working style and areas of responsibility differ. For example, Benjamin made fewer commits but still had a similar amount of line changes to others. The lack of commits is also due to an error with his account was registering commits under the wrong email, so they show up under his GitHub account.

Phuong's code changes are smaller in volume, but she has done more work on the report writing and setting up the database, email system, and CI/CD workflow, which isn't reflected in the above metrics. Every metric is an imperfect representation of the work everyone has done, but these metrics give us a good idea of the overall level of code contributions to the project.

9.8. Evaluation

The system meets the majority of its requirements and is verified with at least 1 test case or quality metric, with 11 of 12 of our functional requirements and 7/7 of non-functional requirements, although NFR6 and NFR7 are only partially covered. Across both, overall requirement coverage is 94.74%. In terms of defects, three were found during testing (DEF-01 to DEF-03) and they are now closed with none of them being of critical severity.

Out of the 12 acceptance criteria, 11 are met and the 12th (FR12, utilisation reports) was pushed to the next sprint. Each of the 11 met criteria is linked to at least one passing test case, so their Given/When/Then behaviour has been checked.

Based on these metrics, the system is close to functionally complete, but they only confirm that each requirement has been tested, not how thoroughly it was tested. The test suite currently covers around 1 test case per requirement; because of this, additional defects may still exist that the current test suite would not detect. Furthermore, line coverage for automated test cases is 33.2%, which is above the 30% CI threshold. However, 30% is not a high metric to pass, so an actual release would likely need a much higher coverage to sufficiently detect defects in the code.

Security is another main risk that lacks test coverage. The tests confirm that students, staff and logged-out users cannot reach the admin pages by URL, but they do not cover direct requests to admin actions, session tampering, input validation such as SQL injection, or weak passwords. Until these are tested, unauthorised access to administrator functions cannot be ruled out. NFR6 (accessibility) and NFR7 (availability) are also only partially covered, as accessibility was tested for language switching only, and availability was observed rather than tested.

9.9. Release Decision

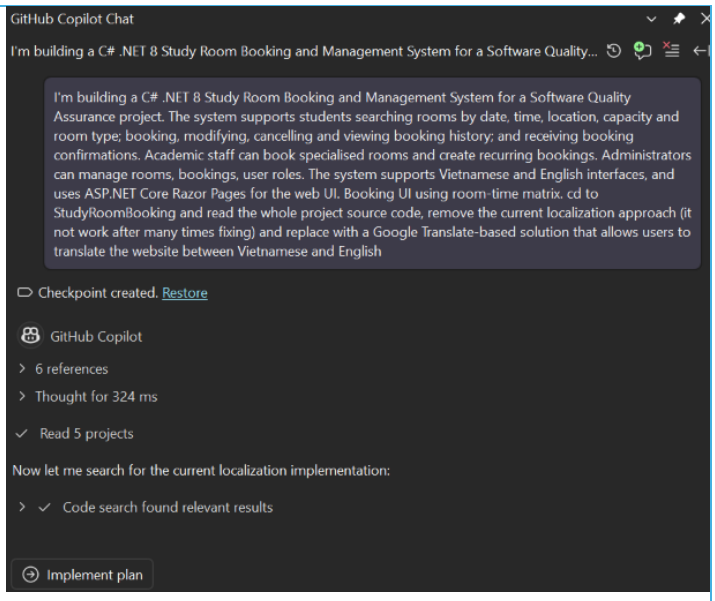
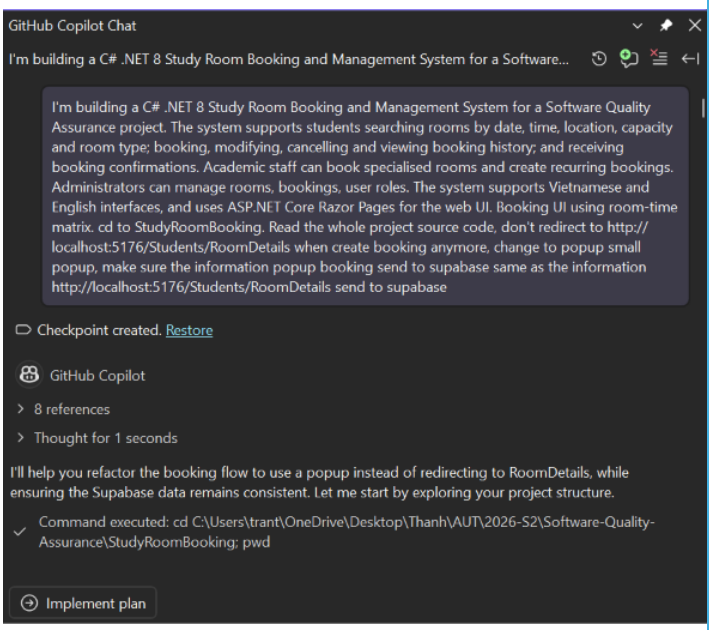
As mentioned above, security testing is limited, FR12 (utilisation reports) is not implemented, and the test suite has not covered enough scenarios to be trusted with real student use. Therefore, the system is not ready for release. The planned improvements needed before release can be seen below in section **11.3**.

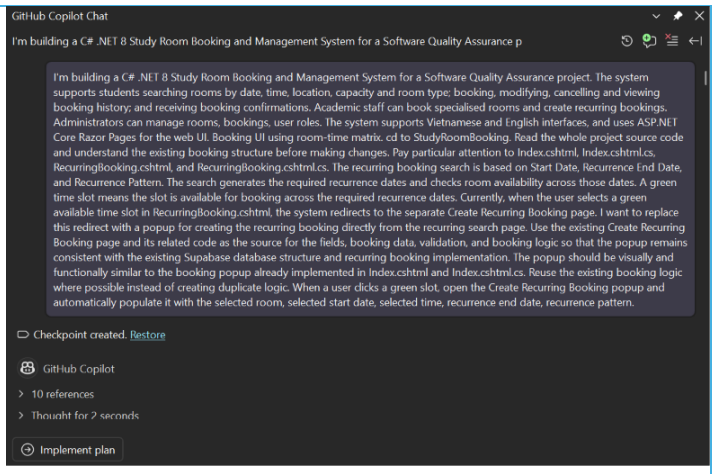
10. AI-Assisted Development and Critical Reflection

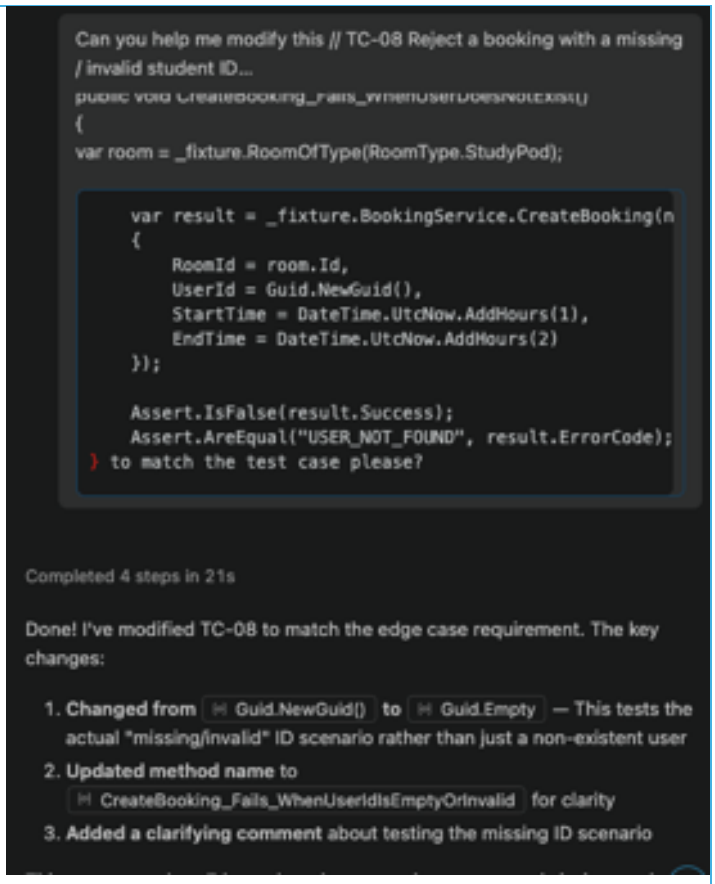
10.1. How GitHub Copilot Was Used

GitHub Copilot was used as an assistant for implementation ideas, debugging, test-case design, CI configuration, and UI refinement. The generated suggestions were reviewed against the requirements and existing code before accepting or modifying them.

Area (date, commits)	How GitHub Copilot was used	Assessment	Evidence
Database and authentication (18 Sep 2026, commit 07b2cb9)	Copilot helped connect the app to Supabase, replace in-memory persistence with Entity Framework repositories, add the database context and unit of work, configure authentication and create the schema	Checked the connection and startup behaviour, and kept the connection string out of source control	Files changed: schema.sql, StudyRoomBookingDbContext.cs, EfRepositories.cs, Program.cs 
Language translation implementation (21–22 Sep 2026, commits c70aba8, 0573ac7)	Copilot assisted with replacing JSON localisation with the Google Translate widget, redesigning the language selector, hiding the default translation banner	Reviewed the selector layout and translation behaviour, then refined the interface and wording to improve usability and clarity for	Files changed: StudyRoomBooking.Infrastructure.csproj, _Layout.cshtml, Program.cs, Login.cshtml, Signup.cshtml

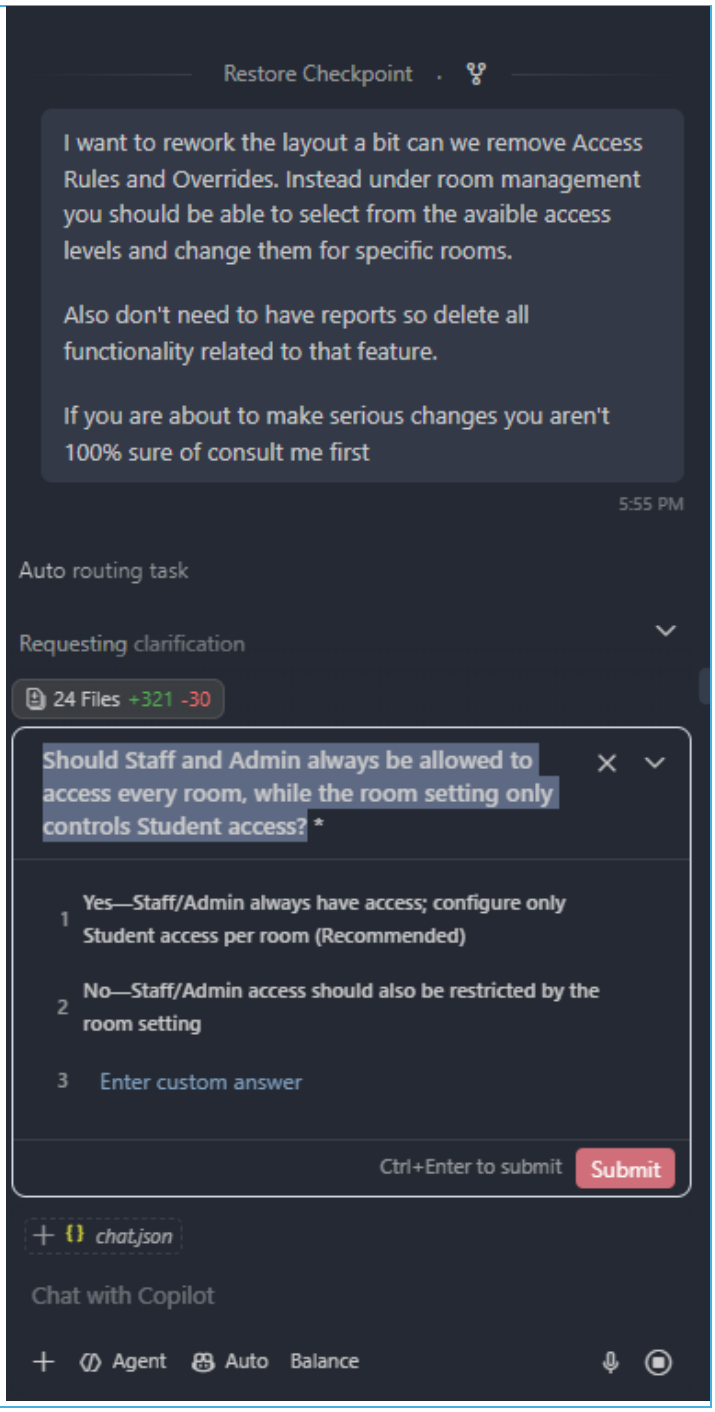
		English- and Vietnamese-speaking users.	
Normal booking search and booking workflow (24–25 Sep 2026, commits 7e79c7e, e62b2ae, 6e156b3, d2b0d9c, 4371ece)	Copilot assisted with adding booking creation and modification popups to the search page, changing the location filter to a dropdown, redesigning search filters for automatic updates, and preventing users from creating overlapping bookings.	Reviewed the search and booking workflow, checked validation rules for conflicting bookings, and refined the interface to make room searching and booking management more convenient.	Files changed: BookingService.cs, Index.cshtml.cs, Index.cshtml, ModifyBooking.cshtml, RoomDetails.cshtml, BuildingLocation.cs, _Layout.cshtml 
Recurrencing booking search and booking workflow (25–27 Sep 2026,	Copilot assisted with implementing recurrence patterns, checking room availability across recurrence dates, displaying booking	Reviewed recurrence date generation, booking availability, time-slot restrictions, and overlapping-	Files changed: BookingService.cs, IBookingService.cs, RecurringBooking.cshtml.cs, RecurringBooking.cshtml, RecurrencePattern.cs, RecurringBookingSearchCriteria.cs, SlotStatus.cs, _ViewImports.cshtml, RoomDetails.cshtml, BuildingLocation.cs, _Layout.cshtml

commits e62b2ae, 4371ece, 03af18e, 86e1a46, f9f54db, a72720a, 702db38)	statuses, and creating a popup to manage recurring bookings. It also helped refine the workflow for creating and deleting bookings and validating conflicts across different rooms.	booking validation. Refined the popup and search interface to improve usability and ensure recurring booking rules were applied correctly.	
Forgot-password UI (26 Sep 2026, commits 7575c56, 529149f, 52ae4fa, 3e2ebe2, and d69591e)	Copilot helped add the entry point and turn the first alert into a styled modal with clearer text	Checked the modal and the responsive login layout before accepting	File changed: Login.cshtml 
MSTest case creation and realistic test data (test-case work	Copilot helped turn requirements into test scenarios and edge cases (invalid dates, conflicts, eligibility, cancellation, history)	Checked each expected result against the service behaviour and used isolated test data so tests	Files changed: BookingServiceTests.cs, StudyRoomBooking.Tests.csproj

recorded in the repository history)		don't affect each other	
CI quality gates (9 October 2026, commits ad0cd0a, c180585, 649079a, b6511b4, 5d2673e, and 5a5d806)	Copilot helped build and debug the GitHub Actions workflow: restore and build, lint, tests with coverage, a 30% threshold, a smoke test	Added the Supabase secret check, passed the connection string through the repository secret after the first smoke test failed, retried, and confirmed the successful run	Files changed: ci.yml, successful CI run evidence

			5. Update GitHub Actions too
			The GitHub secret must also use the Npgsql key/value format, not the postgresql:// URI format.
			Update the repository secret:
			with a value shaped like:
			The workflow runs on:
			`push: branches: [main]`
			`pull_request: branches: [main]`
			7. Check the workflow result
			On GitHub:
			1. Open the repository.
			2. Select **Actions**.
			3. Select the **CI** workflow.
			4. Open the latest run.
			5. Open the `build-and-test` job.
			6. Open the `smoke-test` job.
			The expected result is:

User-secret and CI configuration guidance (included in the database and CI work)	Copilot identified the configuration shape: ConnectionStrings:Supabase through user-secrets or SUPABASE_CONNECTION_STRING, mapped in CI to ConnectionStrings__Supabase	Confirmed the app reports the missing key clearly, with no secret in code or report	Files changed: Program.cs, ci.yml
			The failure is not caused by the compiler warnings. The important error is:
			`ConnectionStrings:Supabase is required`
			The GitHub Actions runner did start the application, but ASP.NET Core did not find the secret in its nested connection-string configuration.
			I updated ci.yml to pass the secret in both supported forms:
			`env: SUPABASE_CONNECTION_STRING: ${{ secrets.SUPABASE_CONNECTION_ ConnectionStrings__Supabase: ${{ secrets.SUPABASE_CONNECTION_`
			`ConnectionStrings__Supabase` is the standard environment-variable form for:
			`ConnectionStrings:Supabase`
			What to do next
			1. Commit the workflow change
			From the repository root:
			`git add .github/workflows/ci.yml`

Administrator functions (19 Sep 2026, commits 3013688, and 4172d97)	Copilot inspected the six admin features, then stored AccessRules and Overrides in the database, added role editing and server-side room validation, and added a shared admin page guard. Copilot later was asked to remove AccessRules, Overrides and Reports and move room access levels into Room Management.	The generated code failed to build three times (a handler inside a try block, a missing using, duplicate variables), so the app was run and fed the errors back. Cancelling booking also crashed until the form was made an explicit POST. Copilot asked whether staff and admin should access every room, because our instruction was unclear.	

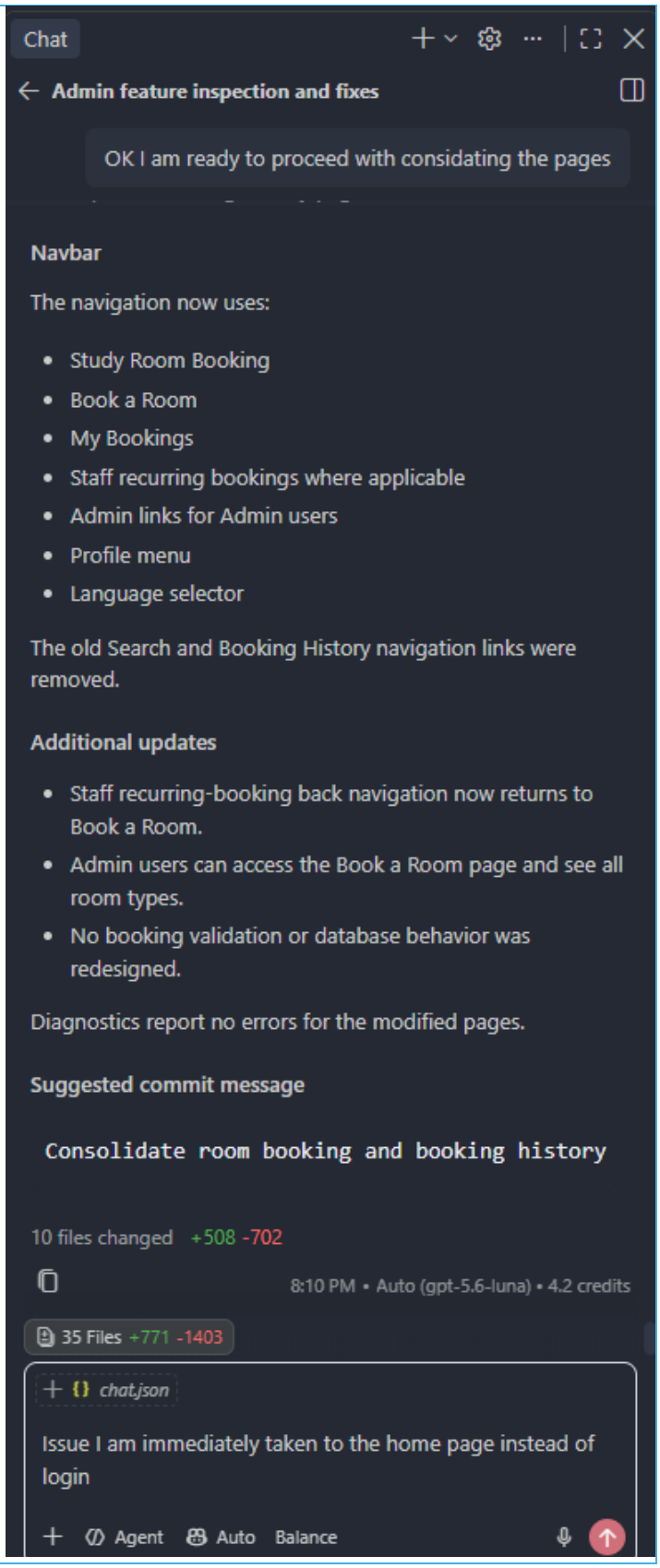
UX/UI rework (19 Sep 2026, commits ac88b86, dc86c3c, c71208c, and d91f814)	Copilot was limited to HTML and CSS and worked in steps, giving a commit message for each. It merged Home and Search into Book a Room, merged My Bookings and Booking History, and split the admin navbar into separate pages	Although only layout changes were requested, users were redirected to the home page instead of the login page. The issue occurred because Copilot had modified the code beyond the requested changes, altering the hyperlink and bypassing the login screen. This was resolved by restoring the hyperlink to its original state.	

Table 29: Use of GitHub Copilot

10.2. How AI Outputs Were Reviewed

For each item:

- The suggestion was compared with the requirements and existing architecture
- The affected services, pages, tests and configuration were reviewed, and the code was adjusted to match the project's structure and naming conventions
- Tests and CI runs were used to verify the changes, with failures and configuration issues recorded rather than assuming the AI-generated output was correct

Outcome	Example
Accepted with changes	Copilot's test scenarios were accepted after correcting expected results to match the real service rules. The CI workflow was accepted after we added the secret check, a lint step and the coverage threshold
Modified	The first forgot-password alert, which became a modal after several rounds of edits to the title, close button and help text
Tested and refined	Recurring booking suggestions were reviewed for recurrence dates, overlapping bookings, time-slot restrictions, and mixed booking statuses. The normal booking workflow was also checked for conflicts and correct modification behaviour
Rejected or reworked	A suggestion that put the connection string in a config file, or a test that used the real database.

Table 30: Review of AI-generated outputs

10.3. Quality Risks Found in AI-Assisted Output

- **Insecure handling of credentials:** The main risk was a connection string ending up in source control or logs. It was stored in user-secrets locally and a repository secret in CI.
- **Unsuitable assumptions about the environment:** The first CI smoke test failed because the app needed a database connection that the workflow did not provide. The generated workflow assumed the app could start with no external configuration (DEF-03).
- **Complex booking rules:** Recurring bookings introduced additional edge cases, including conflicts across multiple rooms, recurrence date limits, and mixed slot statuses. AI-generated changes required careful review and testing to ensure these cases were handled correctly.
- **Design that was not tested with real users:** The original layout was AI-generated and spread features across many pages. Search was combined with Bookings, and My Bookings was combined with History, to reduce steps.

- **Weak or misleading evidence:** Passing tests only prove the tested behaviour. The tests use in-memory SQLite, the lint check only reports analyser errors, and coverage is held at 30%, so passing CI does not mean the code is fully tested or secure.
- **Wording without context:** The English “Free” read as “free of charge” in Vietnamese. A native speaker only found this in manual testing (DEF-02).
- **Outdated or wrong API usage:** An AI-suggested test method (Assert.ThrowsException) did not compile with MSTest 4, which uses ThrowsExactly. This showed the suggestion matched older library versions.

10.4. Reflection

- **Productivity:** AI saved time on repetitive and unfamiliar work, such as test scaffolding, the EF repository setup and the workflow YAML. A first version came quickly, but fixing it took longer than we expected. The smoke test needed several commits to get right.
- **Learning:** The team learned more from investigating why a suggestion failed, such as the secret not reaching the smoke test, than when it worked the first time. Reviewing the output meant that the team had to understand authentication, repositories, CI jobs and coverage, making it easier to explain them during the viva.
- **Testing:** AI was good at suggesting edge cases that might have been missed, but it could not decide what the correct behaviour was. That decision must be based on the requirements. Automation caught logic errors, and manual testing caught usability and language problems.
- **Software quality:** AI-generated code often worked for the main path but did not consider configuration, security and real-world conditions. Quality gates (tests, lint, coverage, smoke test, review) were what made the output trustworthy, not the AI itself.
- **Professional practice:** AI is a useful assistant, but the developer is responsible for the result. Good practice is to review every suggestion, keep secrets out of code, record where AI was used, and verify with tests and a second person’s review.

11. Quality Improvement and Future Recommendations

11.1. Main Quality Lessons Learned

- **Regression testing matters most after architectural change:** Migrating from in-memory repositories to a database was the riskiest change in the project. Rerunning the original booking tests, plus TC-20 for double-booking prevention, showed that the core rules still held. Without a repeatable test suite, this check would have been slow and easy to skip.
- **Automated and manual testing find different problems:** The 12 automated tests covered repeatable logic such as booking, access control and performance. The 8 manual tests found issues that automation could not, such as the English word “Free” being read as “free of charge” in Vietnamese (DEF-02). A passing automated test shows the logic is correct, but not that users understand the interface.
- **Unit tests do not show the whole system works:** All unit tests could pass while the application still failed to start because of configuration or database connection problems (DEF-03). The CI smoke test closed part of this gap.
- **Test environments differ from real ones:** The performance and automated tests used an isolated in-memory SQLite database, and the manual performance test ran on localhost. Both are useful but cannot fully represent the real database or a deployed system.
- **Requirements need testable wording and honest coverage:** Rewriting vague requirements into measurable ones (for example “within 2 seconds for 50 reservations”) made testing possible. The traceability matrix also showed gaps clearly: FR12 was deferred, NFR6 keyboard navigation had no test, and NFR5 and NFR7 had no discrete test case.
- **Quality practices need to be adopted early:** The CI workflow and pull request review were set up late in the project, and earlier changes went straight to main. Starting these from the first sprint would have caught problems sooner and given a stronger record of the work.

11.2. Recommended Improvements to the Development and QA Process

Area	Current situation	Recommendation
Workflow	Pull requests and review were adopted late. This happened due to the project requirements	Use feature branches, pull requests and review from the first sprint, with a pull request template and checklist
Defect tracking	Defects were described in the report, not tracked as they happened	Log every defect as a GitHub Issue with severity, priority and linked pull request, so the register is built from real records
Requirements	Some NFRs have no direct test (NFR5, NFR7)	Define measurable checks for each NFR when requirements are written, for example a maintainability metric and an uptime check

Test planning	Manual tests were designed late	Design test cases alongside requirements, and add acceptance tests before implementation
Reviews	One approving review is required	Add a review checklist covering validation, security, and wording of user-facing text

Table 31: Recommended improvements to the development and QA process

11.3. Future Improvements

Quality Area	Recommended Improvements
Testing	• Run the automated tests against the same database engine as the application, using a test database or containers, instead of in-memory SQLite. • Add concurrency tests for booking, since two students booking the same slot at once is the main reliability risk. • Add end-to-end browser tests for the main flows: search, book, modify, cancel. • Add the missing test for keyboard navigation (NFR6). • Add more test cases per requirement as the current test suite is too limited for release.
Automation and CI/CD	• Raise the coverage threshold from 30% in steps as tests are added, for example to 60% and then 70%. • Strengthen the lint step to a full dotnet format style check, not only analyser errors. • Use a separate test database for the smoke test, so a Supabase outage or data change cannot cause a false failure. • Extend the smoke test to check a login and a booking, not only the home page. • Add automatic deployment to a staging environment once the system is hosted.
Monitoring	• Add application logging and health checks, and track errors and response times after deployment, so quality is measured after release as well as before it. • Measure real search performance and concurrent-user load once deployed, as noted in Section 8.
Security	• Test the threats the current tests do not cover direct requests to admin actions, session tampering, input validation (such as SQL injection) and weak passwords.

	• Add dependency vulnerability scanning and secret scanning to the pipeline. • Do a security review before real students use the system.
Usability and accessibility	• Test usability with more first-time users than the current limited group. • Check accessibility properly: keyboard navigation, screen reader support, colour contrast. • Have native speakers review all translated text and keep a glossary of key terms such as “Available” and “Booked”.
Maintainability	• Keep booking rules in one service and keep the modular structure under review. • Add static analysis and code quality measures and document the architecture and how to run the tests.
Feature completion	Complete FR12 (utilisation reports) with its own test cases and confirm real email delivery with a monitored mailbox or a mail testing service.

Table 32: Future Quality Improvement Recommendations by Quality Area

11.4. Evolving Towards Continuous Quality Engineering

The project already has the start of a quality pipeline: automated tests, CI gates, a smoke test, code review, and a defect and root cause process. A stronger practice would build on it in stages:

1. Short term: make the existing gates stricter (coverage, linting), track defects as issues, and use pull requests for every change.
2. Medium term: test against the real database engine, add browser-level end-to-end tests, and add security scanning to the pipeline.
3. Longer term: deploy to a staging environment automatically, monitor the live system, and use that data (errors, response times, user feedback) to decide what to test and fix next.

The main change is moving from checking quality at the end to building it into every change. Each pull request would be built, tested and scanned automatically, reviewed by a person, and judged against measurable thresholds that the team raises over time.

12. Conclusion

This project continued the development of the Study Room Booking and Management System from Assessment 1, improving its functionality, database integration, usability, and quality assurance practices. The final implementation supports room searching, booking, modification, cancellation, recurring bookings, access control, booking history, email notifications, and English and Vietnamese language switching. However, FR12, room utilisation reporting, was deferred to a future sprint.

Testing covered 20 test cases, including 12 automated tests and 8 manual tests. All 20 passed during final execution. Performance testing recorded an average room search response time of 721 ms with 98 reservations, meeting the two-second target under local test conditions. Security testing also confirmed that unauthorised users could not access the tested administrative pages. However, the results have limitations as automated tests use an isolated in-memory SQLite database, line coverage is 33.2%, and several security, accessibility, and end-to-end scenarios remain untested.

The project also demonstrated the value of combining automated testing, manual testing, regression testing, defect management, code review, and GitHub Actions CI. AI-assisted development helped accelerate implementation and debugging, but its outputs required careful review and verification to ensure that the outputs met the project requirements.

Overall, the project made substantial progress in functionality and software quality. Future work should prioritise completing FR12, expanding automated and end-to-end test coverage, testing against the application's actual database engine, and improving security and accessibility testing. These improvements would help strengthen the system's reliability, usability, and maintainability.

Appendix A: Team Contribution Statement

Thanh Tran – 24247169

Contribution Statement

- **Coding:**
  I was primarily responsible for implementing English and Vietnamese language support and improving the normal and recurring booking workflows and UI.

  For language support, I replaced JSON-based localisation with the Google Translate widget, redesigned the language selector, integrated it across the application pages, such as the login, sign up, and search pages. I also hid the default Google Translate banner and refined the English wording to improve the accuracy and clarity of Vietnamese translations.

  For both normal and recurring booking, I improved the room search interface by removing the Search button and made the search automatically updating search result whenever search criteria changed. I replaced the location text field with a dropdown, added date navigation buttons, improved the room-time matrix, highlighted the current user's bookings in blue with links to modify popup when clicking on, and improved room details and booking statuses legend. I also implemented past-time restrictions, a two-hour booking limit for each booking, and validation to prevent overlapping bookings. Additionally, I added loading indicators in booking, modification, and deletion buttons when clicking on, and disabled controls when updating search result and slot status to prevent unintended actions.

  For normal bookings, I replaced navigation from the search page to the booking creation and modification pages with popup-based workflows. Users can now click a time slot in the matrix to create, modify, or cancel a booking in popup, reducing page navigation and making booking management more convenient.

  For recurring bookings, I added additional recurrence patterns to the search criteria in the search panel. I also refined the creation, modification, and deletion workflows and replaced page navigation with popup-based interactions to make booking management more convenient. Additionally, I implemented conflict warnings for recurring booking and added a "Book Anyway" option, allowing users to book available dates while skipping time slots that overlap with other users' bookings.

  In addition, I implemented the sign-up page, added a Terms and Conditions section during registration, and updated booking history to display expired bookings with an "Expired" status.

- **Writing report:**
  I wrote:
  - Section 1: Introduction
  - Section 4: Test Execution and Final Test Results (task 3)
  - Section 7: Defect Management and Root Cause Analysis (task 6)
  - Part of section 10: AI-Assisted Development and Critical Reflection (task 9)
  - Section 12: Conclusion

I also edited the formatting of the whole report, including standardising headings, fonts, text alignment, as well as captions for tables and figures to ensure consistent presentation throughout the report.

Evidence of Contribution

Evidence of my contribution can be found in GitHub commits, source code changes, and the implemented booking workflows. Relevant commits include:

- Language support: Replaced JSON-based localisation with the Google Translate widget to support English and Vietnamese. Redesigned the language selector, integrated it into the application pages, and hid the default Google Translate banner. Also improved English wording to make Vietnamese translations clearer. (commit c70aba8, 0573ac7, and 5020745)
- Sign up page: Added the sign up page and a Terms and Conditions popup to the sign up page. (commit fce0181, and a3e79a0)
- Normal booking interface and search: Redesigned the room search filters by removing the Search button and start/end time filters, enabling automatic result updates when search criteria changed. Replaced the location text field with a dropdown, added date navigation buttons, and improved the room-time matrix by removing unnecessary status text, highlighting the current user's bookings in blue, linking bookings to modification, expanding the room details area, and displaying the matrix when loading recurring bookings. (Commits: e62b2ae, 4371ece, fdab817, 34b4034)
- Booking creation, modification, and validation: Replaced separate-page navigation with popup-based booking creation and modification directly from the search page. Added validation to prevent users from creating overlapping bookings and restricted booking of time slots within the current hour on the current date. I also fixed normal booking issues involving time selection, modification details, and calendar slot updates. (Commits: 7e79c7e, d2b0d9c, 6e156b3, f07b1bb, f9f54db)
- Recurring booking search and availability: Redesigned the recurring booking search panel, removed unnecessary search and time controls, enabled automatic updates when inputs changed, added recurrence pattern filters, changed the Monthly pattern to FourWeeks, and greyed out past time slots on the current date. Implemented recurrence date generation for None, Daily, Weekly, BiWeekly, and FourWeeks, with availability checks and statuses distinguishing available slots, other users' bookings, the current user's bookings, mixed slots, and unavailable slots. Added recurrence date limits and prevented selection of conflicting time slots. (Commits: 03af18e, 86e1a46, eb85445)
- Recurring booking management and conflict handling: Implemented popup-based recurring booking creation, modification, and deletion workflows, including recurrence date statuses and actions to book missing dates or delete existing bookings. Redesigned the popup interface, enforced two-hour booking slots, and added overlap validation across different rooms. Fixed conflicts involving other

users' booked a slot in the recurrence date slots by displaying warnings and providing a “Book Anyway” option that books available dates while skipping overlap slots of other user. (Commits: a72720a, 702db38, 22c3c83)

- Booking usability, status updates, and bug fixing: Improved booking usability by automatically setting the search start date to the next day after 22:00, disabling interactions while search results were updating, and adjusting recurring booking date validation. Fixed mixed-slot status transitions, recurring date display issues, popup errors, and missing recurrence date statuses. Added loading indicators for booking and deletion operations to provide feedback during processing. (Commits: c22646b, 3a0215b, b27638d, a115162)
- Booking history and expiry: Automatically changed bookings that had passed their scheduled times and had not been cancelled to “Expired”, and displayed them in Previous Bookings. (Commit 3012b46)
- Testing and authorisation: Added booking test cases and cancellation authorisation checks to improve verification of booking behaviour and access control. (Commit 91e998a)

Further evidence includes changes to Login.cshtml, BookingService.cs, IBookingService.cs, RecurringBooking.cshtml.cs, RecurringBooking.cshtml, Index.cshtml, Index.cshtml.cs, and related booking models and view files.

Individual Reflection

Working on normal and recurring bookings taught me the importance of planning the application before implementing it. In Assignment 1, users had to navigate to separate pages to create or modify bookings. Then I found this approach inconvenient, so I changed it to popup-based workflows. However, this change required significantly more effort than the initial implementation as it required many changes to related code and is highly prone to errors during changing process. I also learned that booking conflicts, past time slots, and recurring dates with different statuses can cause unexpected issues if they are not considered carefully. Fixing these bugs helped me understand the importance of testing features together, not just individually. In future, I would consider usability and possible changes earlier in the design stage to reduce rework and avoid introducing new bugs.

Implementing English and Vietnamese language support showed me that there can be different ways to implement the same feature. I initially used JSON-based localisation, but after spending a long time implementing as well as fixing, it still did not work, I switched to Google Translate as an easier option. However, I could not fully control the translated wording, so I improved the original English text to make Vietnamese translations clearer. This experience taught me to consider alternative solutions earlier instead of spending too much time trying to fix an approach that is not working.

AI-assisted development helped me work faster when implementing features and fixing bugs. However, I learned that AI-generated code does not always work as expected and may cause problems when integrated with existing features. I still needed to understand the code, review it, and test it myself. In future, I would plan features more carefully,

write test cases earlier, and test different scenarios after each major change. I would also review AI-generated code before integrating it to reduce bugs and improve the overall quality of the application.

Thi Mai Phuong Nguyen – 23197030

Contribution Statement

- **Coding:**
  I was primarily responsible for the database integration, authentication setup, email confirmation and the CI quality pipeline. I connected the application to Supabase, replaced the in-memory persistence with Entity Framework repositories, added the database context and unit of work, configured authentication, and created the database schema. I implemented the booking confirmation email required by FR5. When a booking is successfully created, the notification service sends an SMTP email to the user's email address. The email includes the booking confirmation number, room name, booking date, start time and end time. I also added configuration for the SMTP host, port, SSL, sender address and credentials, with the email settings supplied through application configuration. The confirmation email also supports the Vietnamese language option. I also improved the login page by adding a Forgot password entry point and turning the first alert into a styled modal with clearer instructions and a close button.
  For quality assurance, I created the GitHub Actions workflow. It restores and builds the solution, runs a lint check, runs the automated tests with TRX and code coverage output, enforces a 30% coverage threshold, and uploads the test results as an artifact. It also runs an HTTP smoke test that starts the web application and checks that it responds successfully using a Supabase connection string stored as a repository secret. I set up the repository so that the connection string is supplied through user-secrets locally and through a repository secret in CI, so no credentials are stored in the source code. The first smoke-test attempt failed because the application did not receive the database connection string correctly. I then updated the workflow to pass the secret in the correct format and retried the test successfully. I also configured a branch protection rule in GitHub to ensure, with every pull request, it will need to pass both two tests and get a review before merging into the main branch.

- **Writing report:**
  Section 3: Requirements Verification and Traceability (Task 2)
  Section 6: Continuous Integration and DevOps QA (Task 5)
  Section 10: AI-Assisted Development and Critical Reflection (Task 9)
  (Note: For Section 10, I wrote the development log and the review process for my individual work on the database integration, forgot-password UI, testing and CI pipeline. I also contributed to the report's explanation of the CI workflow, the use of secrets, the test and coverage quality gates, and the recommendations for a real test database and additional concurrency testing)
  Section 11: Quality Improvement and Future Recommendations (Task 10)

Appendix A: Team Contribution and Professional Reflection (Task 11)
Task 12: Project Presentation and Demonstration (Task 12)

Evidence of Contribution

- Database and authentication: Connected the application to Supabase, added Entity Framework repositories, the database context, unit of work, Supabase authentication, authentication configuration and the database schema.
  Commit: **07b2cb9**
  Files: *StudyRoomBookingDbContext.cs, EfRepositories.cs, EfUnitOfWork.cs, SupabaseAuthClient.cs, AuthenticationService.cs, Program.cs, appsettings.json and schema.sql.*
- Email confirmation: Implemented the booking confirmation email using the notification service and SMTP configuration. The email contains the recipient's booking confirmation number, room name, booking date, start time and end time. I also added a test verification that the notification service is called with the expected booking details.
  Commits:
  - **febc9bb**: Created the booking confirmation email feature and added the notification-service verification to the booking test
  - **4881084**: Updated the email confirmation sender address
  - **e19a798**: Added the Vietnamese booking confirmation version
  Fields: *NotificationService.cs, INotificationService.cs, BookingService.cs, Program.cs, appsettings.json and BookingServiceTests.cs.*
- Forgot password interface: Added the Forgot password entry point and replaced the first alert with a styled modal with clearer text and a close button.
  Commits: **7575c56, 529149f, 52ae4fa, 3e2ebe2 and d69591e**
  File: *Login.cshtml*
- CI quality gates: Created and refined the GitHub Actions workflow with restore and build, lint, automated tests, TRX output, code coverage, a 30% coverage threshold and artifact upload.
  Commits:
  - **ad0cd0a** - Configured the Supabase repository secret for the CI smoke test.
  - **c180585** - Added and ran the CI smoke-test step.
  - **649079a** - Passed the Supabase connection string to the CI smoke test.
  - **b6511b4** - Retried and corrected the CI smoke test.
  - **5d2673e** - Added lint and coverage quality gates.
  - **5a5d806** - Verified the successful CI pull request.
  File: *.github/workflows/ci.yml.*
- Smoke test and secret handling: Added the HTTP smoke test and configured the application to use the Supabase connection string from a GitHub repository secret. The initial smoke-test attempt failed because the connection string was not passed to the application. **649079a** corrected the secret passing, and **b6511b4** retried the smoke test after the correction. **Ad0cd0a** added the repository secret configuration, while **c180585** added the smoke-test step to the workflow.
- Testing contribution: Added the booking confirmation notification verification to the MSTest booking scenarios. The test checks that the notification service

receives the correct email address, room name, booking date, start time, end time and generated confirmation number.
Commit: **febc8bb**
File: *BookingServiceTests.cs*

- Code review setup: I used pull requests and code review for the CI and report work. Pull request #1, verify successful CI pull request, was approved by Benjamin before it was merged. I did set up the branch-protection rule for this one.

Individual Reflection

- The part of this project that taught me the most was getting the CI pipeline to work. I expected the workflow to take a few minutes, but the smoke test failed several times before it passed. It showed me that unit tests passing does not mean the application works. The first run failed because the app needed a database connection that the workflow did not have, and it took me a few commits to pass the secret in the right form. After that I understood why keeping credentials in secrets matters and how the same configuration has to work locally and in CI.
- Working on the database migration showed me why regression testing matters. Moving from in-memory storage to a real database was the biggest change in the project, and rerunning the exiting booking tests, plus the double-booking test after the migration, gave us confidence that the core rules still worked. I also saw that our tests use in-memory SQLite, which is not the same as the real database, so passing tests do not prove everything. This is why I recommended a real test database and concurrency tests in Section 11.
- Using GitHub Copilot and an AI chat assistant made me faster at starting things, such as the repository setup, the test scenarios and the workflow file, but I could not trust any of it without checking. I had to read every suggestion, run it, and fix what was wrong. The risks I noticed most were credentials ending up in code and assumptions that the app needed no configuration. I learned that the developer is responsible for the result, and the quality gates, tests, and code review are what make AI-assisted work reliable.
- Working in the group, I would start pull requests and code review from the first sprint instead of the end. Having them earlier would have given us a better record of the work and caught problems sooner. I worked with Benjamin and Thanh by coordinating the database, application configuration, UI and testing changes. Benjamin's testing and persistence work helped verify that the database migration did not break the booking rules, while Thanh's booking and UI changes had to remain compatible with the authentication, notification and CI configuration. This showed me that teamwork is not only about completing separate features, but also about checking that those features integrate correctly across the whole application.

Benjamin Macpherson – 23208226

Contribution Statement

My contributions towards the projects code were developing several admin features, reworking the information architecture of the website layout and creating and applying a design system to the site. See the evidence of contribution for more details on this.

For the report I wrote sections:

- Section 2. Project Completion and Final Implementation (Task 1)
- Section 5. Test Automation and Quality Engineering (Task 4)
- Section 8. Quality Testing (Task 7)
- Section 9. Quality Metrics and Final Release Decision (Task 8)
- Section 10. AI-Assisted Development and Critical Reflection (Task 9), added the Copilot evidence for the admin features and UI rework

Evidence of Contribution

- Admin features (room management, booking management, user role changes, and merging access rules and overrides into room management): related commits (3013688, 4172d97)
- Website architecture and layout improvements for usability, including the login and admin navigation, merged booking and history pages, and updated navbar headings: related commits (ac88b86, dc86c3c, 5533e80)
- Design system and UI styling, including the colour theme and consistency fixes: related commits (c71208c, d91f814, be8d5b0)
- Test automation, integrating the test cases with an in-memory SQLite database: related commits (c926d85)
- Quality testing for Task 7 (performance and security) and its write-up: related commits (e398127, 7effb5d)
- Report writing for Tasks 1 and 4, quality metrics, and the evaluation and release decision: related commits (5dc6589, a28a7b6, 599afe6, 90c7e9f, 83563d1)
- Code maintenance, fixing compiler warnings and removing a file that broke Git in PowerShell: related commits (d2d5bc5, 9916552)

Individual Reflection

My main takeaway was learning through practice. Carrying out the testing towards the end of the project helped me understand how quality practices such as CI/CD, smoke testing and acceptance testing come together to support a release decision. I was already sure we weren't ready for release, but having quality metrics to back up that reasoning made my points much stronger, and I would feel more confident explaining that decision under pressure in a real development environment.

Developing with Copilot also showed me the importance of clear acceptance criteria. When I asked it to do something without clear criteria for what the code needed to match, it would get parts wrong. In future, I think it will be important for me to learn more about security testing, because this project showed me how much our tests did not cover. If I understood better where data could be breached, we could design our code and test cases to better account for it.